

Multi-key Fully Homomorphic Encryption with Non-Interactive Setup in the Plain Model

Seonhong Min¹, Jeongeun Park², and Yongsoo Song¹

¹ Seoul National University

² Norwegian University of Science and Technology

Abstract. Multi-key fully homomorphic encryption (MKFHE) enables computation over encrypted data under multiple different keys. Constructing MKFHE without any trusted or interactive setup remains an open problem. In the context of MKFHE, a trusted setup is often assumed to mean the use of a common random string (CRS).

In this paper, we present the first MKFHE scheme in the plain model (i.e., without any trusted or interactive setup) based solely on the RLWE assumption. Our design yields a 2-round multi-party computation (MPC) in the plain model against semi-malicious adversaries. Moreover, it can be applied to transform existing FHE schemes that rely on RGSW in their construction into a multi-key variant. We also provide concrete conversions for widely-used FHE schemes, including BGV, BFV, CKKS, FHEW, TFHE, and Carousel.

Finally, we implement our scheme and present experimental results for the expansion algorithm from a single-key ciphertext to a multi-key ciphertext and the multi-key homomorphic multiplication algorithm.

1 Introduction

Over the last decade, fully homomorphic encryption (FHE), which enables computation over encrypted data, has emerged as a promising solution for privacy-preserving computation. Its non-interactive nature makes it particularly well-suited for cloud-based applications, enabling secure processing of sensitive data without requiring interaction between parties [25,46,26,34,24]. Notably, recent advances have brought FHE to a level of practicality where it is not only adopted in research prototypes but also deployed in real-world systems, for example, Apple’s FHE-based caller ID system [53].

Despite recent promising progress, most of the development has focused on the single-user setting, whereas real-world cloud-based applications often involve data from multiple owners. Examples include collaborative analysis, federated learning, joint financial risk assessments, and secure multi-party healthcare data sharing. Recognizing the importance of this field, the U.S. National Institute of Standards and Technology (NIST) recently issued a call [55] to promote research on extending FHE to support multi-party settings.

Multi-key fully homomorphic encryption (MKFHE) [44] directly supports computation across multiple parties without requiring interaction between participants. In other words, MKFHE enables computation on inputs encrypted

under different keys, allowing a function to be evaluated over multiple parties' data while ensuring that each party's input remains secret from the others.

Such a strong functionality of MKFHE can be applied to a wide range of real-world scenarios, including blockchain platforms. In fact, FHE has been applied to blockchain in works to enable privacy-preserving smart contracts. However, all of these constructions are designed for the single-user setting [54] or rely on a global secret key distributed to trusted MPC nodes [62]. Therefore, to support multi-user computation in a truly decentralized manner, it is essential to achieve MKFHE without a trusted setting.

Indeed, the first ever construction of MKFHE [44], namely the LTV scheme, requires no interaction among parties and does not rely on any shared randomness, such as a common random/reference string (CRS), which is generated from *a trusted setup*. This feature was mainly due to the employment of the DSPR (Decisional Small Polynomial Ratio) problem, which is a milder variant of the NTRU problem. On the other hand, its disadvantage lies in the multi-round decryption, which stems from the multiplicative joint key structure. This issue was later resolved by Ananth, Jain, Jin, and Malavolta, from a generic transformation for constructing MKFHE in the plain model (without a trusted setup), which supports a one-round decryption protocol [4]. The transformation requires the following three building blocks:

1. An MKFHE scheme with a trusted setup and one-round decryption,
2. An MKFHE scheme without a trusted setup but with an arbitrary round decryption,
3. A two-round semi-malicious oblivious transfer protocol without a trusted setup.

Concretely, the first building block can be instantiated easily with existing RLWE-based MKFHE schemes in the CRS model [16,17,37,39], as they inherently support one-round decryption. The LTV scheme can be used for the second building block, and finally, the third block can be constructed from LWE-based two-round OT protocols, for instance, [10,7].

Later, however, the DSPR problem was found to be insecure when scaled to larger parameters [19,1], making the LTV scheme vulnerable, particularly in scenarios involving a large number of keys in the MKFHE setting. Therefore, subsequent developments in this field have shifted toward employing a different hardness assumption, namely Ring Learning with Errors (RLWE), in order to avoid the known attacks and allow more flexible parameter choices.

Nevertheless, all secure constructions [18,48,37,39,16,17] based on RLWE rely on a CRS to achieve practical efficiency and simplify key entanglement for correct decryption. While there have been a few attempts [11,35] to remove reliance on a CRS by introducing an interactive setup among a fixed number of parties, this approach sacrifices the dynamic property, namely, the ability to support ad hoc participants on-the-fly, which was inherently supported by the original MKFHE construction. Several recent works [59,50,57] attempt to revive DSPR in the context of MKFHE with carefully chosen small parameters to ensure security, but all of them ultimately rely on (R)LWE ciphertexts for correctness, thereby

requiring a CRS. Furthermore, the scheme proposed in [59] was later shown to be insecure due to its use of a fixed Hamming weight, as demonstrated in [36].

Consequently, it leads us to the following fundamental research question:

Can we construct a secure MKFHE scheme in the plain model without any pre-computation, and with one-round decryption?

Importantly, even under a relaxed version of the problem where one-round decryption is not required and it suffices to support an arbitrary number of decryption rounds, no existing *secure MKFHE scheme* without a trusted setup is known to fulfill this requirement ³.

Our Contribution. In this paper, we resolve the research question by constructing a multi-key variant of Gentry-Sahai-Waters fully homomorphic encryption scheme [29], which we call multi-key RGSW, based solely on RLWE in the plain model with no interactive setup. As a direct implication of a transform from [4], we achieve one-round decryption as well.

In previous papers, the use of a CRS appears in RGSW ciphertexts, which consist of vectors of RLWE ciphertexts. For example, an RGSW ciphertext can be written as $(\mathbf{b}, \mathbf{A})$, where $\mathbf{A}$ is a random matrix of polynomials in $\mathbb{Z}_Q[X]/(X^N + 1)$ for some positive integer Q and power-of-two N , and it is assumed to be a CRS shared among the parties.

In contrast, we construct a multi-key RGSW scheme without a common polynomial matrix, such as $\mathbf{A}$. This results in a multi-key FHE scheme that does not rely on a CRS. In more detail, at the core of our MKFHE, there lies a new single-key encryption called the *compact LEV* (cLEV), which offers a multiplicative operation between a vector of ring elements and an encrypted message. We extend the cLEV cryptosystem to a multi-key variant of the RGSW cryptosystem via homomorphic operations in Section 3. Then we show that our multi-key RGSW cryptosystem satisfies the definition of MKFHE in Section 4. However, since the entanglement is no longer linear, decryption lacks the clean structure typically seen in CRS-based MKFHE schemes.

As a result, the server must explicitly follow the non-linear relation of the joint key⁴ for correctness, without learning its actual value. In our solution, the server follows this relation by keeping track of the order of multiplications, which poses little overhead in an outsourced computation setting where the circuit information is already known. This tracked information is used by the server for key switching and bootstrapping, which performs decryption homomorphically to enable unlimited operations.

After the desired homomorphic computation, the decryption can be executed by evaluating the decryption circuit of the RGSW scheme using any existing MPC primitives. Nevertheless, we demonstrate that we can construct an

³ Note that the transform from [4] directly implies a one-round MKFHE as a result.

⁴ The joint key, which is a function of the individual secret keys, is not known to any party, as it is generated through homomorphic operations. Each party knows the function structure and their own secret share, but not the exact value of the joint key.

MKFHE scheme in the plain model with one-round decryption via the transformation by Ananth et al. [4]. While we do not explicitly construct a one-round decryption protocol in combination with the other components, our result provides the final missing ingredient required for its theoretical realization. Our work thus bridges a critical gap and opens the way for the future concrete instantiation of MKFHE in the plain model with one-round decryption. We note that this furthermore yields a two-round multi-party computation (MPC) in the plain model against semi-malicious adversaries, as established in [4]⁵. A theoretical motivation for MKFHE is to enable MPC protocols with very low round complexity. While prior works [11,35] require at least three rounds, our result achieves the optimal round complexity. We describe a concrete protocol based on our MKFHE in Section 4.3.

In addition, we show that our scheme can be applied to existing RLWE-based schemes such as BFV [9,28], CKKS [20], TFHE [21], LMKCDEY [41], and Carousel [33], as we discuss in Section 5. These RLWE-based FHE schemes employ the RGSW scheme to perform multiplicative operations, and our design almost directly implies the multi-key variant of these schemes. In other words, our multi-key RGSW ciphertext can replace any CRS-based RGSW ciphertext, thereby eliminating the use of a CRS in existing schemes.

Finally, we provide a proof-of-concept implementation of our MKFHE scheme using the HIENAA.jl library [32] with concrete parameter sets and demonstrate the performance of our scheme in Section 6. For example, a single multiplication takes 243, 985, and 6964 milliseconds when the number of parties is 2, 4 and 8, respectively.

Scheme	Security Model	Hardness Assumption	Non-interactive Setup	Secure?
[44]	Plain	DSPR	✓	×
[11,35]	Plain	(R)LWE	×	✓
[48,23,51,13,18,42] [16,17,37,39,58]	CRS	(R)LWE	✓	✓
[59]	CRS	DSPR, (R)LWE	✓	×
[57,50]	CRS	DSPR, (R)LWE	✓	✓
This Work	Plain	(R)LWE	✓	✓

Table 1. Comparison with related works. The last column indicates whether there is a known attack which breaks the underlying hardness assumption in the context of MKFHE.

In Table 1, the security model, hardness assumption, and communication model of each multi-key homomorphic encryption scheme are summarized. As shown in the table, our scheme is the only one that (1) is constructed in the

⁵ Since our design satisfies all the assumptions and conditions of their paper, we do not provide a separate security proof.

plain model, (2) relies solely on the RLWE hardness assumption, (3) requires no setup communication, and (4) has no known attacks against its underlying assumption.

1.1 Technical Overview

For some positive integer $Q > 0$ and a power-of-two N (which is a fixed parameter throughout the paper), we write $R := \mathbb{Z}[X]/(X^N + 1)$ and $R_Q := R/QR$. Also, $\mathbf{g} = (g_0, \dots, g_{d-1}) \in \mathbb{Z}^d$ is a gadget vector, and $h : R_Q \rightarrow R^d$ is the corresponding decomposition function, such that $\|h(a)\|_\infty$ is small and $a = h(a)^\top \cdot \mathbf{g} \pmod{Q}$ for any $a \in R_Q$.

Compact Leveled Encryption. Consider a multi-key RLWE ciphertext $\mathbf{c} = (c_0, \dots, c_{k-1}) \in R_Q^{1 \times k}$ of a plaintext $m \in R_Q$ under a joint key $\mathbf{sk} = (s_0, \dots, s_{k-1}) \in R^{k \times 1}$ of parties $1, \dots, k-1$. In other words,

$$\sum_{i=0}^{k-1} c_i s_i = \mathbf{c} \cdot \mathbf{sk} \approx m \pmod{Q}.$$

We remark that the plaintext m is encrypted in a noisy manner, which we neglect in this overview. In later sections, we discuss how one can keep the noise level small during the homomorphic computation, ensuring the correct decryption result. Now, assume that the k -th party wants to multiply some plaintext $\mu \in R_Q$ with this ciphertext. In the conventional RLWE-RGSW multiplication, the linear decryption formula is homomorphically computed, but the secret is multiplied by the plaintext. Specifically, we homomorphically evaluate

$$\sum_{i=0}^{k-1} c_i \cdot (\mu \cdot s_i) = \mathbf{c} \cdot (\mu \cdot \mathbf{sk}) \approx \mu \cdot m \pmod{Q}$$

given the encryptions of $g_j \cdot \mu \cdot s_i$ for $0 \leq i < k$ and $0 \leq j < d$. However, this is challenging to realize in the conventional multi-key scenario, as the parties do not have access to all s_i 's and therefore cannot generate the encryption of $g_j \cdot \mu \cdot s_i$'s directly. In prior works [48,51,16,39], this challenge was resolved by using the CRS. Roughly speaking, the encryptions of $g_j \cdot \mu \cdot s_i$ can be homomorphically generated from the pre-computed arithmetic relation between the CRS and each party's key.

Instead, we homomorphically compute a vector $\mu \cdot \mathbf{c} \in R_Q^{1 \times k}$ packed in a single ciphertext. Our approach does not require any information on the other parties' secrets, removing the need for additional arithmetic relations using CRS. However, the challenge here is that a conventional multi-key ciphertext can encrypt only a single ring element, not a vector. Therefore, we need a new cryptosystem that can encrypt a plaintext vector into a single ciphertext. For this, we utilize the shared-A RLWE. Briefly speaking, the shared-A RLWE is identical to several RLWE ciphertexts that share the randomness part, encrypted under different keys. To be specific, given the secret key $\mathbf{t} = (t_1, \dots, t_k) \in R^{k \times 1}$, a

shared-A RLWE encryption of a vector $\boldsymbol{\mu} = (\mu_1, \dots, \mu_k) \in R_Q^{1 \times k}$ is given by $(b_1, \dots, b_k, a) \in R_Q^{1 \times (k+1)}$ where

$$\forall 1 \leq i \leq k, \quad b_i + a \cdot t_i \approx \mu_i \pmod{Q}.$$

It is worth noting that the shared-A RLWE has been explicitly and implicitly considered in many works [52,12,5,6], including MKFHE [16,17,37,39].

On top of shared-A RLWE, we construct a new encryption scheme, called the *compact leveled encryption*, cLEV for short. Let k be a positive integer, the dimension of the encryption. Concretely, a cLEV encryption of μ of dimension k is composed of shared-A RLWE encryptions of each row of the matrix $\mu \cdot \mathbf{g} \otimes \mathbf{I}_k$, where $\otimes$ denotes the tensor product of matrices and $\mathbf{I}_k$ denotes the identity matrix of the size $k \times k$. Consequently, a cLEV encryption of dimension k is a matrix of ring elements of the size $kd \times (k+1)$, where the i -th row is a shared-A RLWE encryption of the i -th row of $\mu \mathbf{g} \otimes \mathbf{I}_k$.

We also propose a new multiplicative operation called the *vector gadget product*, defined between a vector of ring elements of size k and a cLEV encryption of dimension k . It homomorphically computes the following linear equation:

$$\begin{aligned} [h(c_0)^\top \ h(c_1)^\top \ \dots \ h(c_{k-1})^\top] \cdot (\mu \mathbf{g} \otimes \mathbf{I}_k) &= [c_0 \ c_1 \ \dots \ c_{k-1}] \cdot (\mu \mathbf{I}_k) \\ &= \mu \cdot \mathbf{c} \pmod{Q} \end{aligned}$$

for the input vector $\mathbf{c} = (c_0, \dots, c_{k-1}) \in R_Q^{1 \times k}$. In ciphertext level, the vector gadget product between $\mathbf{c} \in R_Q^{1 \times k}$ and a cLEV ciphertext $\mathbf{C} \in R_Q^{kd \times (k+1)}$ of dimension k , which we denote by $\mathbf{c} \boxtimes_v \mathbf{C}$, is as follows:

$$\mathbf{c} \boxtimes_v \mathbf{C} = [h(c_0)^\top \ h(c_1)^\top \ \dots \ h(c_{k-1})^\top] \cdot \mathbf{C} \pmod{Q}.$$

Now, assume that $\mathbf{c}$ is a multi-key encryption of m under the secret $\mathbf{sk}$. Then, the resulting ciphertext of the vector gadget product can be seen as an encryption of $\mu \cdot m$ under a ‘joint’ secret of $\mathbf{sk}$ and $\mathbf{t}$. Let us delve into detail. Observe that the decryption of a shared-A RLWE encryption $(b_1, \dots, b_k, a) \in R_Q^{1 \times (k+1)}$ of a vector $(\mu_1, \dots, \mu_k) \in R^{1 \times k}$ under the secret $\mathbf{t} = (t_1, \dots, t_k) \in R^{k \times 1}$ can be written as the following linear equation:

$$[b_1 \ \dots \ b_k \ a] \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{t}^\top \end{bmatrix} \approx [\mu_1 \ \dots \ \mu_k] \pmod{Q}.$$

This directly follows from the definition of shared-A RLWE. Being a stack of shared-A RLWE encryptions, a cLEV encryption $\mathbf{C} \in R_Q^{kd \times (k+1)}$ of $\mu \in R$ is decrypted with a linear relation, as follows:

$$\mathbf{C} \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{t}^\top \end{bmatrix} \approx \mu \cdot \mathbf{g} \otimes \mathbf{I}_k \pmod{Q}.$$

Therefore, for the output ciphertext $\mathbf{c}' = \mathbf{c} \boxtimes_v \mathbf{C} \in R_Q^{1 \times (k+1)}$ of the vector gadget product, we have the following equation:

$$\mathbf{c}' \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{t}^\top \end{bmatrix} \cdot \mathbf{sk} = [h(c_0)^\top \ h(c_1)^\top \ \dots \ h(c_{k-1})^\top] \cdot \mathbf{C} \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{t}^\top \end{bmatrix} \cdot \mathbf{sk}$$

$$\begin{aligned}
&= [h(c_0)^\top \ h(c_1)^\top \ \dots \ h(c_{k-1})^\top] \cdot (\mu \mathbf{g} \otimes \mathbf{I}_k) \cdot \mathbf{sk} \\
&\approx \mu \cdot \mathbf{c}^\top \cdot \mathbf{sk} \approx \mu \cdot m \pmod{Q}.
\end{aligned}$$

The above relation implies that the vector gadget product outputs a ciphertext encrypted under a joint key $\begin{bmatrix} \mathbf{I}_k \\ \mathbf{t}^\top \end{bmatrix} \cdot \mathbf{sk} = \begin{bmatrix} \mathbf{sk} \\ \mathbf{t}^\top \cdot \mathbf{sk} \end{bmatrix}$. In our scenario, each i -th party outputs cLEV encryption of dimension i and performs the vector gadget product sequentially in order to compute a product of their messages. More specifically, $\mathbf{C}_i \in R_Q^{id \times (i+1)}$ be a cLEV ciphertext of dimension i encrypting some message $\mu_i \in R_Q$ under $\mathbf{s}_i \in R^i$, the secret key of the i -th party for $1 \leq i \leq n$, where n denotes the number of participants. Then, for some plaintext $m \in R_Q$, $m \cdot \prod_{i=1}^n \mu_i \in R_Q$ can be homomorphically computed as $\mathbf{c} := (\dots ((m \boxtimes_v \mathbf{C}_1) \boxtimes_v \mathbf{C}_2) \boxtimes_v \dots) \boxtimes_v \mathbf{C}_n$, encrypted under the secret $\overline{\mathbf{sk}} := \begin{bmatrix} \mathbf{I}_n \\ \mathbf{s}_n^\top \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_{n-1} \\ \mathbf{s}_{n-1}^\top \end{bmatrix} \dots \begin{bmatrix} \mathbf{I}_1 \\ \mathbf{s}_1^\top \end{bmatrix}$. We remark that the final joint secret has the form $\overline{\mathbf{sk}} = (1, \bar{s}_1, \dots, \bar{s}_n) \in R^{(n+1) \times 1}$ for some polynomials $\bar{s}_i \in R$ for $1 \leq i \leq n$.

Multi-key RGSW. The cLEV encryption and vector gadget product enable multiple parties to compute the product of private data, however, its functionality is still limited as it only allows the parties to multiply once and no further homomorphic multiplications. Therefore, it is important to achieve full homomorphism, i.e., an unlimited number of homomorphic multiplications and additions. We achieve this by extending the cLEV encryption into a multi-key RGSW ciphertext.

Previously, the joint secret of the resulting ciphertext was given by $\overline{\mathbf{sk}} = (1, \bar{s}_1, \dots, \bar{s}_n)$. To facilitate RGSW-RGSW multiplication under this secret, we require MLWE encryptions of each entry of $\mu \cdot (\mathbf{g} \otimes \overline{\mathbf{sk}})$, i.e., MLWE encryptions of $g_j \mu \bar{s}_i$ and $g_j \mu$ for $1 \leq i \leq n$ and $0 \leq j < d$. Our core observation is that $\bar{s}_i$ is essentially a multivariate polynomial of $s_{j,k}$'s, where $s_{j,k}$ denotes the k -th entry of the secret $\mathbf{s}_j$ of j -th party, for $1 \leq j \leq n$ and $0 \leq k < j$.

Without loss of generality, assume that μ belongs to the i -th party. Naïvely, we can compute all the terms of each multivariate polynomial $g_m \mu \bar{s}_\ell$ using the cLEV encryptions of $s_{j,k}$'s for $1 \leq i \neq j \leq n$ and $0 \leq k < j$, $\mu \cdot s_{i,k}$ for $0 \leq k < i$, and finally μ , then add them together to obtain an MLWE encryption of $g_m \mu \bar{s}_\ell$ for $0 \leq m < d$ and $1 \leq \ell \leq n$. However, this naïve approach may require an unnecessarily large amount of vector gadget multiplications, leading to poor performance. Instead, we propose a circuit-like approach that can inductively generate each row of RGSW encryptions.

Let $\overline{\mathbf{sk}}_i := \prod_{j=1}^i \begin{bmatrix} \mathbf{I}_j \\ \mathbf{s}_j^\top \end{bmatrix}$. Then, it is easy to see $\overline{\mathbf{sk}}_{i+1} = \begin{bmatrix} \overline{\mathbf{sk}}_i \\ \mathbf{s}_{i+1}^\top \cdot \overline{\mathbf{sk}}_i \end{bmatrix}$, hence

$$\overline{\mathbf{sk}}_{i+1} = \begin{bmatrix} \overline{\mathbf{sk}}_i \\ \mathbf{s}_{i+1}^\top \cdot \overline{\mathbf{sk}}_i \end{bmatrix}. \quad (1)$$

The core intuition of the circuit approach is based on this relation. Assume that μ belongs to the i -th party. Our goal is to generate the RLWE encryptions of

$g_j \mu$ and $g_j \mu \cdot \bar{s}_i$ under secret $\bar{\mathbf{sk}}$, for $1 \leq i \leq n$ and $0 \leq j < d$. To allow this, we first compute all the encryptions of $g_j s_i$ under the key $\mathbf{sk}_{\text{idx}-1}$ for all $1 \leq i < \text{idx}$, iteratively. Computing $g_j \cdot \bar{s}_1$ is straightforward, as $\bar{s}_1 = s_{1,0}$. Now, suppose that we are given encryptions of $g_j \bar{s}_1, g_j \bar{s}_2, \dots, g_j \bar{s}_i$ for some $1 \leq i < \text{idx}$ under the key $\mathbf{sk}_i$. Recall that $\bar{s}_{i+1} = \sum_{k=0}^i \bar{s}_k s_{i+1,k}$ from Eq. (1). Therefore, we perform $i+1$ vector gadget multiplications between the RLWE encryption of $g_j \cdot \bar{s}_k$ under $\mathbf{sk}_i$ and the cLEV encryption of $s_{i+1,k}$ for $0 \leq k \leq i$ under $\mathbf{s}_{i+1}$ to obtain encryptions of $g_j \bar{s}_k s_{i+1,k}$ under the secret $\mathbf{sk}_{i+1}$, then the summation of them is essentially the RLWE encryption of $g_j \bar{s}_{i+1}$ under $\mathbf{sk}_{i+1}$. The mathematical induction implies that we can generate the RLWE encryptions of $g_j \bar{s}_i$ for $1 \leq i < \text{idx}$ under the secret $\mathbf{sk}_{\text{idx}-1}$.

Next, we compute the encryptions of $g_j \mu \bar{s}_i$ for $1 \leq i \leq \text{idx}$ analogously, using the cLEV encryptions of μ and $\mu s_{\text{idx},j}$ for $0 \leq j < \text{idx}$. Specifically, if $i < \text{idx}$, we can multiply the cLEV encryption of μ to the encryptions of $g_j \bar{s}_i$, and if $i = \text{idx}$, compute $\bar{s}_{\text{idx}} = \sum_{k=0}^{\text{idx}} \bar{s}_k s_{\text{idx},k}$.

Finally, given the RLWE encryptions of $g_j \mu \bar{s}_i$ for all $1 \leq i \leq \text{idx}$ under secret $\mathbf{sk}_{\text{idx}}$, we can compute the encryptions of $g_j \mu \bar{s}_i$ under secret $\mathbf{sk}_n$ easily by inductively performing homomorphic evaluation of the equation $g_j \mu \bar{s}_{i+1} = \sum_{k=0}^i g_j \mu \bar{s}_k \cdot s_{i+1,k}$. In total, we perform $n \cdot n(n+1)/2 = O(n^3)$ vector gadget products to obtain an RGSW encryption of some message μ .

Formalizing Our Multi-key RGSW. We formally describe our novel multi-key RGSW scheme, algorithm by algorithm, in the multi-key syntax. We first describe the key generation and the encryption algorithm, along with a ciphertext shrinking method for the cLEV encryption. Suppose that we are given a cLEV encryption $\mathbf{C} = (c_{i,j})_{0 \leq i < dK, 0 \leq j \leq K} \in R_Q^{dK \times (K+1)}$ of μ under the secret $\mathbf{t} = (t_i)_{0 \leq i < K} \in R^{K \times 1}$. Then, for $\mathbf{C}' = (c'_{i,j})_{0 \leq i < dK', 0 \leq j \leq K'} \in R_Q^{dK' \times (K'+1)}$ and $\mathbf{t}' = (t'_i)_{0 \leq i < K'}$ for some $1 \leq K' < K$, it is easy to see that

$$\mathbf{C}' \cdot \begin{bmatrix} \mathbf{I}_{K'} \\ \mathbf{t}'^\top \end{bmatrix} \approx \mu \mathbf{g} \otimes \mathbf{I}_{K'},$$

hence making $\mathbf{C}'$ a cLEV encryption of dimension K' under the secret $\mathbf{t}'$. Now, recall that our multi-key RGSW scheme requires the parties' secrets to have different dimensions. Therefore, in a direct instantiation of our scheme, the key generation and encryption algorithms depend on the index of the ciphertext and, hence, are inconsistent among the parties. To resolve this issue, we set the maximum dimension K and fix the key generation and encryption algorithms to output the secret keys and cLEV encryptions of dimension K . Then, the evaluator can receive the cLEV ciphertexts from different parties, choose the evaluation order, and proceed with the evaluation in that order while converting the ciphertext to the right dimension.

We also describe the evaluation algorithm. When there are multiple evaluators, their joint keys may differ as they can choose different multiplication orders. To allow computation between the multi-key RGSW ciphertexts under different joint keys, we propose an efficient key-switching key generation algorithm. Given

two joint secrets $\overline{\mathbf{sk}}, \overline{\mathbf{sk}}' \in R^{(n+1) \times 1}$, a key-switching key from $\overline{\mathbf{sk}} = (1, \overline{s}_1, \dots, \overline{s}_n)$ to $\overline{\mathbf{sk}}'$ is composed of encryptions of $g_j \cdot \overline{s}_i$ under $\overline{\mathbf{sk}}'$ for $1 \leq i \leq n$ and $0 \leq j < d$. To generate this, we first generate RGSW encryptions of $s_{i,j}$'s, j -th entry of i -th party's key $\mathbf{s}_i \in R^K$ under $\overline{\mathbf{sk}}'$ using the RGSW extension algorithm. Then, the recurrence relation Eq. (1) can be homomorphically evaluated with the RGSW encryptions of $s_{i,j}$'s.

Finally, we describe how the distributed decryption can be instantiated with a generic MPC protocol and explain how our scheme can be extended to a semi-malicious security setting with non-interactive zero-knowledge proofs (NIZKs). Our distributed decryption relies on the transformation of Ananth et al. [4], which allows us a one-round decryption for multi-key schemes with any non-linear decryption formula.

Universal Key Generation. Although RGSW is theoretically useful due to its structural design, which does not require any evaluation keys other than the ciphertexts themselves, its performance makes it difficult to use as a main cryptographic tool in real-world applications. Therefore, we discuss how to apply our construction of the multi-key RGSW scheme to the best-performing FHE schemes, including CKKS [20], BFV [9,28], TFHE [21], Carousel [33] and LMKCDEY [41], thereby allowing us to compute with those schemes in the plain model.

The main challenge for MKFHE schemes lies in the evaluation key generation. Especially for the CKKS or BFV schemes, the generation of the relinearization and automorphism keys is the most crucial part.

For the joint secret $\overline{\mathbf{sk}} = (1, \overline{s}_1, \dots, \overline{s}_n) \in R^{(n+1) \times 1}$, the automorphism key for some automorphism Ψ is given by a stack of the encryptions of $g_i \cdot \Psi(\overline{s}_j)$ under $\overline{\mathbf{sk}}$ for $0 \leq i < d$ and $1 \leq j \leq n$. To generate this, we use the Eq. (1) again: we iteratively compute $\Psi(\overline{s}_{i+1}) = \sum_{k=1}^i \Psi(\overline{s}_k) \Psi(s_{i+1, i-k})$ for $1 \leq i \leq n$, given the cLEV encryptions of $\Psi(s_{i,j})$ for all $1 \leq i \leq n$ and $0 \leq j < i$. On the other hand, the relinearization key can be seen as RGSW encryptions of $\overline{s}_i$'s under the secret $\overline{\mathbf{sk}}$, for $1 \leq i \leq n$. An easy way to achieve this is to generate all the RGSW encryptions of $s_{i,j}$ under $\overline{\mathbf{sk}}$, and use the same core formula $\overline{s}_{i+1} = \sum_{k=0}^i \overline{s}_k s_{i+1, k}$ as the ciphertext extension to inductively generate the RGSW encryptions of $\overline{s}_i$ for $1 \leq i \leq n$.

In contrast, the main challenge for the AP-like MKFHE schemes, namely TFHE [21], Carousel [33] and LMKCDEY [41], lies in the blind rotation key generation. Suppose that each coefficient of $\overline{\mathbf{sk}} = (1, \overline{s}_1, \dots, \overline{s}_n)$ is contained in an interval $[-B, B]$ for some integer bound $B > 0$. Then, the blind rotation key is given by the RGSW encryptions of binary values that indicate if a coefficient of $\overline{s}_i$ is a certain value inside $[-B, B]$. In other words, for $\overline{s}_{i,j}$, j -th coefficient of $\overline{s}_i$, we generate the RGSW encryptions of $\delta(\overline{s}_{i,j} + B), \dots, \delta(\overline{s}_{i,j} - B)$ where δ denotes the Dirac delta function. We observe that these encryptions can be generated iteratively from the ‘indicator form’ of the coefficients of each party’s secret.

2 Preliminary

2.1 Notations

For a power-of-two N , $R := \mathbb{Z}[X]/(X^N + 1)$ be the $2N$ -th cyclotomic ring of integers, and $R_Q := R/QR$. We do not specify N when N is consistent in the context. An element in R , or R_Q is represented with a polynomial with degree less than $N - 1$, in other words, $a(X) = \sum_{i=0}^{N-1} a_i X^i$. $\delta_{a,b}$ denotes the delta function which outputs 1 if and only if $a = b$, and 0 otherwise. $\mathbf{I}_k$ denotes the identity matrix of the size $k \times k$. For a polynomial a (either in R or R_Q), $\|a\|_\infty$ denotes the infinity norm of the coefficient vector of a . $\mathbf{M}^\top$ denotes the transpose of a matrix $\mathbf{M}$.

2.2 Multi-key FHE

A multi-key fully homomorphic encryption (MKFHE) is an encryption scheme that can evaluate an arbitrary circuit over encrypted data. We follow the definition by Lopez-Alt et al. [44], where the algorithms are dependent on a parameter K , the number of distinct secret keys that the scheme can deal with.

Definition 1 (Multi-key Fully Homomorphic Encryption). *A multi-key fully homomorphic encryption (MKFHE) scheme is a family of probabilistic polynomial-time (PPT) algorithms $\{\mathcal{E}^{(K)} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Eval}, \text{Dec})\}_{n>0}$, defined as follows.*

- $\text{pp} \leftarrow \text{Setup}(1^\lambda, 1^K)$: On input a security parameter λ and the number of parties K , returns a public parameter pp .
- $(\text{sk}, \text{ek}, \text{pk}) \leftarrow \text{KeyGen}(\text{pp})$: On input a public parameter pp , returns a secret key, (public) evaluation key, and public key pair $(\text{sk}, \text{ek}, \text{pk})$.
- $\text{ct} \leftarrow \text{Enc}(\text{pk}; m)$: On input a public key pk and a message m , returns a ciphertext ct .
- $\text{ct} \leftarrow \text{Eval}(\mathcal{C}, \{(\text{ek}_i, \text{pk}_i, \text{ct}_i)\}_{i \in [\ell]})$: On input the circuit $\mathcal{C}$ and ℓ tuples $(\text{ek}_i, \text{pk}_i, \text{ct}_i)$ where each consists of an (public) evaluation key ek_i , a public key pk_i and a ciphertext ct_i , returns a ciphertext ct .
- $\mu \leftarrow \text{Dec}(\{\text{sk}_i\}_{i \in [K]}; \text{ct})$: On input secret keys $(\text{sk}_i)_{i \in [K]}$ and a ciphertext ct , returns a message m .

An MKFHE scheme needs to satisfy correctness and compactness. Formally, the following properties hold for all integers $K > 0$, circuits $\mathcal{C}$, all sequences of K key tuples $\{(\text{ek}'_i, \text{pk}'_i, \text{sk}'_i)\}_{i \in [K]}$ each of which is in the support of $\text{KeyGen}(1^\lambda)$, all sequences of ℓ key tuples $\{(\text{ek}_i, \text{pk}_i, \text{sk}_i)\}_{i \in [\ell]}$ each of which is in $\{(\text{ek}'_i, \text{pk}'_i, \text{sk}'_i)\}_{i \in [K]}$, and all messages $(m_i)_{i \in [\ell]}$ and ciphertexts $(\text{ct}_i)_{i \in [\ell]}$ such that ct_i is in the support of $\text{Enc}(\text{pk}'_i; m_i)$.

- **Correctness:** For $\text{ct} \leftarrow \text{Eval}(\mathcal{C}, \{(\text{pk}_i, \text{ct}_i)\}_{i \in [K]})$, $\text{Dec}(\{\text{sk}_i\}_{i \in [K]}; \text{ct}) = \mathcal{C}(m_1, \dots, m_K)$.
- **Compactness:** For $\text{ct} \leftarrow \text{Eval}(\mathcal{C}, \{(\text{pk}_i, \text{ct}_i)\}_{i \in [K]})$, there exists a bivariate polynomial P such that the size of ct is bounded by $P(\lambda, K)$, hence independent from the depth of the evaluated circuit $\mathcal{C}$.

2.3 Ring Learning With Errors

We describe the ring variants of the decisional Learning With Errors (LWE) problem, namely the Ring-LWE (RLWE) and Module-LWE (MLWE) problems. The RLWE [56,45] and MLWE [40] problems are defined over the ideal lattice, especially over the ring of integers of a cyclotomic field, below.

Definition 2 (Decisional RLWE). *Let Q be positive integers, N be a power-of-two, and ψ be an error distribution over $R := \mathbb{Z}[X]/(X^N + 1)$. The Ring Learning with Errors (RLWE) distribution with parameters N, Q, ψ is defined as follows: sample $a \xleftarrow{\$} R_Q$ and $e \leftarrow \psi$, and output (b, a) where $b = -as + e \pmod{Q}$ for the secret $s \in R$. For a key distribution χ over R , the decisional RLWE problem, denoted by $\text{RLWE}(N, Q, \psi, \chi)$ calls for distinguishing the RLWE distribution for a secret $s \leftarrow \chi$ from the uniform distribution over R_Q^2 .*

Definition 3 (Decisional MLWE). *Let k, Q be positive integers, N be a power-of-two, and ψ be an error distribution over $R := \mathbb{Z}[X]/(X^N + 1)$. The Module Learning With Errors (MLWE) distribution with parameters k, N, Q, ψ is defined as follows: sample $\mathbf{a} \xleftarrow{\$} R_Q^{1 \times k}$ and $e \leftarrow \psi$, and output $(b, \mathbf{a}) \in R_Q^{1 \times (k+1)}$ where $b = -\langle \mathbf{a}, \mathbf{s} \rangle + e \pmod{Q}$ for the secret $\mathbf{s} \in R^k$. For a key distribution χ over R , the decisional MLWE problem, denoted by $\text{MLWE}(k, N, Q, \psi, \chi)$, calls for distinguishing the MLWE distribution for a secret $\mathbf{s} \leftarrow \chi^k$ from the uniform distribution over $R_Q^{1 \times (k+1)}$.*

We also remark that the RLWE problem is simply an extreme case of the MLWE problem, with $k = 1$.

In many works [52,12,5,6], a variant of RLWE, called the shared-A RLWE, that considers multiple (R)LWE samples with shared (public) randomness, was studied for better efficiency. We describe the shared-A RLWE as follows.

Definition 4 (Decisional shared-A RLWE). *Let k, Q be positive integers, N be a power-of-two, and ψ be an error distribution over $R := \mathbb{Z}[X]/(X^N + 1)$. The shared-A Ring Learning With Errors (shared-A RLWE) distribution with parameters k, N, Q, ψ is defined as follows: sample $c_0 \xleftarrow{\$} R_Q$ and $e_i \leftarrow \psi$ for $1 \leq i \leq k$, and output $(c_0, c_1, \dots, c_k) \in R_Q^{1 \times (k+1)}$ where $c_i = -c_0 \cdot s_i + e_i \pmod{Q}$ for $1 \leq i \leq k$, for the secret $\mathbf{s} = (s_1, \dots, s_k) \in R^k$. For a key distribution χ over R , the decisional shared-A RLWE problem, denoted by $\text{sARLWE}(k, N, Q, \psi, \chi)$, calls for distinguishing the shared-A RLWE distribution for a secret $\mathbf{s} \leftarrow \chi^k$ from the uniform distribution over $R_Q^{1 \times (k+1)}$.*

Based on these hardness problems, we can define the MLWE and shared-A encryption schemes.⁶ An MLWE encryption of a plaintext $\mu \in R_Q$ under the

⁶ We exclude the RLWE encryption, as it is essentially an extreme case of the MLWE encryption.

secret $\mathbf{s} \in R^k$ is given by a tuple $(b, \mathbf{a}) \in R_Q^{1 \times (k+1)}$ such that $b + \langle \mathbf{s}, \mathbf{a} \rangle \approx \mu \pmod{Q}$. For convenience, we also say that the secret of this ciphertext is $\mathbf{sk} = \begin{bmatrix} 1 \\ \mathbf{s}^\top \end{bmatrix} \in R^{(k+1) \times 1}$, since $(b, \mathbf{a}) \cdot \mathbf{sk} = b + \langle \mathbf{s}, \mathbf{a} \rangle \approx \mu \pmod{Q}$. On the other hand, a shared-A encryption of a plaintext vector $(\mu_1, \dots, \mu_k) \in R_Q^{1 \times k}$ under the secret $\mathbf{s} = (s_1, \dots, s_k) \in R^k$ is given by a tuple $(b_1, \dots, b_k, a) \in R_Q^{1 \times (k+1)}$ such that $b_i + as_i \approx \mu_i \pmod{Q}$ for $1 \leq i \leq k$. We remark that these relations can be written in a single linear transformation, as follows.

$$\begin{bmatrix} b_1 & \dots & b_k & a \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{s} \end{bmatrix} \approx \begin{bmatrix} \mu_1 \\ \vdots \\ \mu_k \end{bmatrix} \pmod{Q}.$$

2.4 Gadget Decomposition and GSW

The gadget decomposition technique is a widely used noise management technique in lattice-based homomorphic cryptography. Let $R := \mathbb{Z}[X]/(X^N + 1)$ for some power-of-two N . Roughly speaking, the gadget decomposition is an injective function $h : R_Q \rightarrow R^d$ which takes an arbitrary ring element modulo Q as an input and returns a small vector over R such that the input can be recovered with a dot product between the output vector and some vector $\mathbf{g} \in R^d$. Here, the vector $\mathbf{g} \in \mathbb{Z}^d$ is called the *gadget vector*, and parameter $d > 0$ is called the *gadget dimension*. Let us formally describe the gadget decomposition below.

Definition 5 (Gadget Decomposition). *Let Q and d be positive integers. Then, an injective function $h : R_Q \rightarrow R^d$ is called the gadget decomposition with respect to the gadget vector $\mathbf{g} \in \mathbb{Z}^d$ if it satisfies the following two conditions, for all $a \in R_Q$ and some $B > 0$.*

- $\|h(a)\|_\infty \leq B$.
- $\langle h(a), \mathbf{g} \rangle = a \pmod{Q}$.

Typically, we choose the gadget decomposition h and the gadget vector $\mathbf{g}$ so that the bound $B \approx Q^{1/d}$. In the rest of the draft, we abuse the notation of the gadget decomposition h for the vector as well; in other words, $h((a_1, \dots, a_k)) = (h(a_1)^\top \mid \dots \mid h(a_k)^\top)^\top$. Similarly, this notation can be extended to the point-wise decomposition of matrices as well.

Building upon the gadget decomposition, we also describe the ring variant of the Gentry-Sahai-Waters (GSW) cryptosystem, referred to as *RGSW*. Let us first describe the encryption and decryption algorithm. Below, we fix the gadget decomposition $h : R_Q \rightarrow R^d$ and the corresponding gadget vector $\mathbf{g} \in \mathbb{Z}^d$, and the noise distribution ψ .

• **RGSW.Enc($\mathbf{s}; \mu$)** : Given the secret $\mathbf{s} \in R^k$ and the plaintext μ , sample $\mathbf{A} \xleftarrow{\$} R_Q^{d(k+1) \times k}$ and $\mathbf{e} \leftarrow \psi^{d(k+1)}$. Output $\overline{\mathbf{C}} = [\mathbf{b} \mid \mathbf{A}] \in R_Q^{d(k+1) \times (k+1)}$ where

$$\mathbf{b} = -\mathbf{A} \cdot \mathbf{s} + \mu \cdot \mathbf{g} \otimes \begin{bmatrix} 1 \\ \mathbf{s} \end{bmatrix} + \mathbf{e} \pmod{Q}.$$

• **RGSW.Dec($\mathbf{s}; \overline{\mathbf{C}}$)** : Given the secret $\mathbf{s} \in R^k$ and the RGSW encryption $\overline{\mathbf{C}} \in R_Q^{d(k+1) \times (k+1)}$, find and output μ that minimizes

$$\mathbf{e} := \overline{\mathbf{C}} \cdot \begin{bmatrix} 1 \\ \mathbf{s} \end{bmatrix} - \mu \cdot \mathbf{g} \otimes \begin{bmatrix} 1 \\ \mathbf{s} \end{bmatrix} \pmod{Q}.$$

The RGSW cryptosystem offers two multiplicative operations in addition to the additive and subtractive operations that are naturally supported by the linearity of RLWE, called the *internal product* [29] and *external product* [21]. Roughly speaking, the internal product takes two RGSW ciphertexts and outputs an RGSW ciphertext encrypting the multiplication of the messages of the input ciphertexts. On the other hand, the external product takes MLWE and RGSW ciphertexts for each and outputs an MLWE ciphertext encrypting the multiplication of the messages of the input ciphertexts. We describe them formally below.

Definition 6 (Internal Product). Given RGSW ciphertexts $\overline{\mathbf{C}}_1, \overline{\mathbf{C}}_2 \in R_Q^{d(k+1) \times (k+1)}$, the internal product $\boxdot : R_Q^{d(k+1) \times (k+1)} \times R_Q^{d(k+1) \times (k+1)} \rightarrow R_Q^{d(k+1) \times (k+1)}$ is defined as follows:

$$\overline{\mathbf{C}}_1 \boxdot \overline{\mathbf{C}}_2 = h(\overline{\mathbf{C}}_1)^\top \cdot \overline{\mathbf{C}}_2 \pmod{Q}.$$

Definition 7 (External Product). Given an MLWE ciphertext $\mathbf{c} \in R_Q^{1 \times (k+1)}$ and an RGSW ciphertext $\overline{\mathbf{C}} \in R_Q^{d(k+1) \times (k+1)}$, the external product $\boxtimes : R_Q^{1 \times (k+1)} \times R_Q^{d(k+1) \times (k+1)} \rightarrow R_Q^{1 \times (k+1)}$ is defined as follows:

$$\mathbf{c} \boxtimes \overline{\mathbf{C}} = h(\mathbf{c})^\top \cdot \overline{\mathbf{C}} \pmod{Q}.$$

We roughly show the correctness of the internal and external products. Let $\mathbf{s}$ be the secret key of the ciphertexts, and m, μ_1, μ_2 be the underlying plaintexts of $\mathbf{c}, \overline{\mathbf{C}}_1, \overline{\mathbf{C}}_2$, respectively. Then,

$$\overline{\mathbf{C}}_1 \boxdot \overline{\mathbf{C}}_2 \cdot \begin{bmatrix} 1 \\ \mathbf{s} \end{bmatrix} \approx h(\overline{\mathbf{C}}_1)^\top \cdot \mu_2 \mathbf{g} \otimes \begin{bmatrix} 1 \\ \mathbf{s} \end{bmatrix} = \mu_2 \cdot \overline{\mathbf{C}}_1 \cdot \begin{bmatrix} 1 \\ \mathbf{s} \end{bmatrix} = \mu_1 \mu_2 \cdot \mathbf{g} \otimes \begin{bmatrix} 1 \\ \mathbf{s} \end{bmatrix} \pmod{Q}.$$

Therefore, $\overline{\mathbf{C}}_1 \boxdot \overline{\mathbf{C}}_2$ is a valid RGSW encryption of the message $\mu_1 \mu_2$. On the other hand,

$$\mathbf{c} \boxtimes \overline{\mathbf{C}}_1 \cdot \begin{bmatrix} 1 \\ \mathbf{s} \end{bmatrix} \approx h(\mathbf{c})^\top \cdot \mu_1 \mathbf{g} \otimes \begin{bmatrix} 1 \\ \mathbf{s} \end{bmatrix} = \mu_1 \cdot \mathbf{c} \cdot \begin{bmatrix} 1 \\ \mathbf{s} \end{bmatrix} = \mu_1 m \pmod{Q}.$$

Hence, $\mathbf{c} \boxtimes \overline{\mathbf{C}}_1$ is an MLWE encryption of the message $\mu_1 m$.

3 Multi-key RGSW without CRS

In this section, we construct a new multi-key cryptosystem, called the compact leveled encryption (cLEV for short), which allows the parties to perform multiplication exactly once in a specific order without CRS or communication. Building upon cLEV, we construct a new multi-key variant of the RGSW cryptosystem, which allows an unlimited number of homomorphic multiplications.

3.1 Compact Levelled Encryption

We begin by formally describing the encryption algorithm of the compact leveled encryption (cLEV) and the vector gadget product, which is a multiplicative operation between a vector of ring elements and a cLEV encryption. Below, ψ denotes the error distribution defined over R . $\mathbf{g}$ and $h : R_Q \rightarrow R^d$ are the gadget vector and the corresponding decomposition function. Finally, k denotes the dimension of the cLEV encryption.

- cLEV.Enc($\mathbf{s}; \mu$) : Given the message $\mu \in R_Q$ and secret key $\mathbf{s} \in R^k$, sample $\mathbf{a} \xleftarrow{\$} R_Q^{kd}$, and $\mathbf{E} \leftarrow \psi^{kd \times k}$. Output

$$[-\mathbf{a} \cdot \mathbf{s}^\top + \mu \cdot \mathbf{g} \otimes \mathbf{I}_k + \mathbf{E} \bmod Q \mid \mathbf{a}] \in R_Q^{kd \times (k+1)}.$$

We remark that the cLEV encryption scheme relies on the shared-A RLWE problem, with parameters k, N, Q, ψ and χ where $\mathbf{s} \leftarrow \chi^k$ for the secret key distribution χ .

Definition 8 (Vector Gadget Product). *Given a vector $\mathbf{c} \in R_Q^{1 \times k}$ and a cLEV ciphertext $\mathbf{C} \in R_Q^{dk \times (k+1)}$ of dimension k , the binary operation $\boxtimes_v : R_Q^{1 \times k} \times R_Q^{dk \times (k+1)}$ is defined as*

$$\mathbf{c} \boxtimes_v \mathbf{C} = h(\mathbf{c})^\top \cdot \mathbf{C} \pmod{Q}.$$

Roughly speaking, the vector gadget product between a vector $\mathbf{c} \in R_Q^{1 \times k}$ and a cLEV encryption $\mathbf{C}$ of message $\mu \in R$ outputs a shared-A encryption of $\mu \cdot \mathbf{c} \in R_Q^{1 \times k}$. Observe that the following holds for the cLEV encryption $\mathbf{C} \in R_Q^{kd \times (k+1)}$ of $\mu \in R$ under $\mathbf{s} \in R^k$.

$$C \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{s}^\top \end{bmatrix} \approx \mu \cdot \mathbf{g} \otimes \mathbf{I}_k \pmod{Q}.$$

Therefore, given $C \in R_Q^{dk \times (k+1)}$, a cLEV encryption of μ under the secret $\mathbf{s} = (s_1, \dots, s_k)$, the following holds for any $\mathbf{c} \in R_Q^{1 \times k}$.

$$(\mathbf{c} \boxtimes_v C) \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{s}^\top \end{bmatrix} = h(\mathbf{c})^\top \cdot C \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{s}^\top \end{bmatrix} \approx h(\mathbf{c})^\top \cdot \mu \cdot \mathbf{g} \otimes \mathbf{I}_k = \mu \cdot \mathbf{c} \pmod{Q}. \quad (2)$$

Now we discuss how to perform a homomorphic multiplication between ciphertexts encrypted under different keys, leveraging the cLEV encryption and the vector gadget product. Assume that n parties are participating in the computation, and each i -th party discloses a cLEV ciphertext of dimension i , encrypting plaintext $\mu_i \in R$ under the secret $\mathbf{s}_i \in R^i$. Then, $((m \boxtimes_v C_1) \boxtimes_v \dots) \boxtimes_v C_{n-1} \boxtimes_v C_n$ is a (noisy) encryption of $m \cdot \prod_{i=1}^n \mu_i$ for any $m \in R_Q$ under the secret

$\begin{bmatrix} \mathbf{I}_k \\ \mathbf{s}_k^\top \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_{k-1} \\ \mathbf{s}_{k-1}^\top \end{bmatrix} \cdots \begin{bmatrix} \mathbf{I}_1 \\ \mathbf{s}_1^\top \end{bmatrix}$. This can be easily shown by applying Eq. (2) multiple times, as follows.

$$\begin{aligned}
& (((m \boxtimes_v C_1) \boxtimes_v \dots) \boxtimes_v C_{n-1}) \boxtimes_v C_n \cdot \begin{bmatrix} \mathbf{I}_n \\ \mathbf{s}_n^\top \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_{n-1} \\ \mathbf{s}_{n-1}^\top \end{bmatrix} \cdots \begin{bmatrix} \mathbf{I}_1 \\ \mathbf{s}_1^\top \end{bmatrix} \\
& \approx \mu_n \cdot ((m \boxtimes_v C_1) \boxtimes_v \dots) \boxtimes_v C_{n-1} \cdot \begin{bmatrix} \mathbf{I}_{n-1} \\ \mathbf{s}_{n-1}^\top \end{bmatrix} \cdots \begin{bmatrix} \mathbf{I}_1 \\ \mathbf{s}_1^\top \end{bmatrix} \\
& \approx \dots \approx \mu_n \mu_{n-1} \dots \mu_1 \cdot m \pmod{q}.
\end{aligned} \tag{3}$$

Since the generation of cLEV ciphertexts $C_1, \dots, C_n$ does not require any additional assumptions (e.g., CRS), the vector gadget product naturally enables the multi-key multiplication in the plain model. We remark that the resulting ciphertext of our multi-key multiplication algorithm has a different joint key structure compared to the existing solutions. Previously, the secret of a multi-key ciphertext was given by $(1, s_1, \dots, s_k) \in R^{k+1}$ where each s_i belongs to the i -th party. In contrast, the joint secret $\begin{bmatrix} \mathbf{I}_k \\ \mathbf{s}_k^\top \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_{k-1} \\ \mathbf{s}_{k-1}^\top \end{bmatrix} \cdots \begin{bmatrix} \mathbf{I}_1 \\ \mathbf{s}_1^\top \end{bmatrix}$ is a multiplication of each party's secret matrix.

3.2 Multi-key RGSW

The multi-key multiplication via the vector gadget product allows us to evaluate arbitrary multivariate polynomials between different parties. This directly comes from the fact that the RLWE cryptosystem naturally supports ciphertext addition. To be specific, given a multivariate polynomial, we can simply compute each term with cLEV encryptions of each party's message and the vector gadget product, and then perform ciphertext addition. Let n be the number of participants, and there exists a multivariate polynomial that is evaluated during the computation. Then, each term of the polynomial can be written as a multiplication of some coefficient and private data from a subset of participants.

Therefore, each party only needs to output the cLEV encryptions of their message and use the vector gadget product to compute each term. While this gives us a functionality of computing arbitrary multivariate polynomials, it still suffers from a few downsides. Firstly, the number of input ciphertexts grows exponentially as the polynomial degree increases. Recall that each party gets to multiply only once per term, making the parties output all the monomials required in the computation. Secondly, it does not support circuit evaluation, which is generally much cheaper than direct computation.

To overcome these issues, we construct a new multi-key variant of the RGSW scheme [29] on top of the cLEV cryptosystem and the vector gadget product. Let $\bar{\mathbf{s}} = (\bar{s}_0, \bar{s}_1, \dots, \bar{s}_n)$ be the joint secret for n parties, obtained from the gadget vector products. Recall that the RGSW encryption of a plaintext $\mu \in R$ under this joint secret is composed with the MLWE encryptions of $g_j \mu \bar{s}_i$ for $0 \leq i \leq n$ and $0 \leq j < d$, and s_i 's are multivariate polynomials of each party's secret key components.

Therefore, we can generate each row of the RGSW encryption with the vector gadget multiplication and ciphertext addition. To allow this, we require the cLEV encryptions of each party's secret key components, and the cLEV encryption of μ and μ multiplied by the secret key components of the encrypting party ⁷. Let us first investigate the joint key structure of our multi-key scheme. Let $\mathbf{s}_i = (s_{i,0}, \dots, s_{i,i-1}) \in R^i$ be the secret of which the cLEV ciphertexts of i -th party are encrypted under, and $\overline{\mathbf{sk}}_i \in R^{i+1}$ be the secret of the resulting RLWE after vector gadget multiplication by the cLEV ciphertext of i -th party. Then, Eq. (3) implies that

$$\overline{\mathbf{sk}}_i = \begin{bmatrix} \mathbf{I}_i \\ \mathbf{s}_i^\top \end{bmatrix} \cdot \overline{\mathbf{sk}}_{i-1} = \begin{bmatrix} \overline{\mathbf{sk}}_{i-1} \\ \mathbf{s}_i^\top \cdot \overline{\mathbf{sk}}_{i-1} \end{bmatrix} \quad (4)$$

for all $i > 1$, where $\overline{\mathbf{sk}}_1 = \begin{bmatrix} \mathbf{I}_1 \\ \mathbf{s}_1^\top \end{bmatrix} = (1, s_{1,1})$. Therefore, for $\overline{\mathbf{sk}} = \overline{\mathbf{sk}}_n = (\overline{s}_0 = 1, \overline{s}_1, \dots, \overline{s}_n)$, we have the recurrence relation

$$\overline{s}_i = \sum_{j=1}^i \overline{s}_{j-1} s_{i,j}. \quad (5)$$

It is easy to see that the number of terms of a multivariate polynomial $\overline{s}_i$ is the sum of the numbers of the terms of each $\overline{s}_0, \dots, \overline{s}_{i-1}$. This is essentially a geometric sequence with a common ratio of two, making the number of terms of $\overline{s}_i$ grow exponentially, 2^{i-1} terms to be precise. Therefore, a naïve evaluation of $g_j \mu \overline{s}_i$ requires 2^{i-1} vector gadget multiplications, making the RGSW extension inefficient with the time complexity exponential to the number of the parties n . Instead, we leverage the recurrence relation to reduce the time complexity for the RGSW extension from $O(2^n)$ to $O(n^2)$.

The main idea is to generate the encryption of $g_j \mu \overline{s}_i$ from the encryptions of $g_j \mu \overline{s}_0, \dots, g_j \mu \overline{s}_{i-1}$. Since $g_j \mu \overline{s}_i = \sum_{k=1}^i g_j \mu \overline{s}_{k-1} \cdot s_{i,k}$, we can iteratively generate the encryptions of $g_j \mu \overline{s}_k$'s and utilize the vector gadget encryption to multiply $s_{i,k}$. We provide the full algorithm in Algorithm 1, that generates an RGSW encryption of μ , which is a secret data from the idx -th party, under the joint key $\overline{\mathbf{sk}}$. In the algorithm, $\text{cLEV}_{\mathbf{s}_i}(\mu)_{j+1}$ denotes the $j+1$ -th row of the cLEV encryption $\text{cLEV}_{\mathbf{s}_i}(\mu)$.

Correctness.

Lemma 1. *Given cLEV encryptions $\text{cLEV}_{\mathbf{s}_i}(s_{i,j})$ of $s_{i,j}$ for $1 \leq i \leq n$ and $1 \leq j \leq i$, and a cLEV encryption $\text{cLEV}_{\mathbf{s}_{\text{idx}}}(\mu)$ of idx -th party, Algorithm 1 outputs an RGSW encryption of μ under the joint secret key $\overline{\mathbf{sk}} = (\overline{s}_i)_{1 \leq i \leq n}$, obtained from the recurrence relation from Eq. (5).*

⁷ We remark that this resembles the structure of the RGSW scheme. The difference is that our scheme is based on the shared-A RLWE, while ordinary RGSW is based on MLWE.

Algorithm 1 RGSW extension

Input: $\text{cLEV}_{\mathbf{s}_i}(s_{i,j})$ for $1 \leq i \neq \text{idx} \leq n$ and $0 \leq j < i$, $\text{cLEV}_{\mathbf{s}_{\text{idx}}}(\mu)$ and $\text{cLEV}_{\mathbf{s}_{\text{idx}}}(\mu s_{\text{idx},j})$ for $0 \leq j < \text{idx}$.

- 1: **for** $0 \leq i < d$ **do**
- 2: $\text{acc}_i^0 \leftarrow (g_i)$
- 3: **end for**
- 4: **for** $1 \leq i \leq n$, $0 \leq j < d$ **do**
- 5: **if** $i \neq \text{idx}$ **then**
- 6: **for** $0 \leq k < i$ **do**
- 7: $\text{acc}_{dk+j}^i \leftarrow (\text{acc}_{dk+j}^{i-1} \mid 0)$
- 8: **end for**
- 9: $\text{acc}_{di+j}^i \leftarrow \sum_{k=0}^{i-1} \text{acc}_{dk+j}^{i-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_i}(s_{i,k})$
- 10: **else**
- 11: $\text{acc}_j^i \leftarrow \text{cLEV}_{\mathbf{s}_i}(\mu)_{j+1}$
- 12: **for** $0 < k < i$ **do**
- 13: $\text{acc}_{dk+j}^i \leftarrow \text{acc}_{dk+j}^{i-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_i}(\mu)$
- 14: **end for**
- 15: $\text{acc}_{di+j}^i \leftarrow \sum_{k=0}^{i-1} \text{acc}_{dk+j}^{i-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_i}(\mu s_{i,k})$
- 16: **end if**
- 17: **end for**

Output: $\bar{\mathbf{C}} = \begin{bmatrix} \text{acc}_0^n \\ \vdots \\ \text{acc}_{d(n+1)-1}^n \end{bmatrix}$

Proof. See Appendix A.

It is worth noting that there are some implicit ideas hidden in our RGSW extension algorithm. Firstly, we switched the secret of a multi-key MLWE ciphertext to a secret with a larger dimension for free. Specifically, we appended a zero at the rightmost entry of the MLWE ciphertext acc_{dk+j}^{i-1} in the inner loop, consequently switching the joint secret from $\overline{\mathbf{sk}}_{i-1}$ to $\overline{\mathbf{sk}}_i$. Secondly, we also showed that a shared-A encryption can be directly used as a multi-key MLWE ciphertext under the joint secret. In more detail, we took a row from a single-key cLEV encryption and directly used it as a multi-key MLWE ciphertext under the joint secret in line 11 of the algorithm. We leverage these ideas in the later sections as well.

3.3 Discussion

Compact GSW (cGSW). Recall Algorithm 1, and observe that the RGSW extension algorithm requires not only a cLEV encryption of the message μ , but also cLEV encryptions of $\mu \cdot s_{\text{idx},j}$'s for $0 \leq j < \text{idx}$. We remark that this resembles the structure of the RGSW encryption, which requires all the encryptions $g_j \mu \bar{s}_i$ for all $1 \leq i \leq n$, not only the encryptions of $g_j \mu$. Therefore, for convenience, we define the compact GSW (cGSW) encryption as follows.

• **cGSW.Enc($\mathbf{s}; \mu$)** : Given the message $\mu \in R_Q$ and secret key $\mathbf{s} = (s_0, \dots, s_{k-1}) \in R^k$, let $\mathbf{C}_0 \leftarrow \text{cLEV.Enc}(\mathbf{s}; \mu)$ and $\mathbf{C}_{i+1} \leftarrow \text{cLEV.Enc}(\mathbf{s}; \mu \cdot s_i)$ for $0 \leq i < k$. Output $\mathbf{C} = \{\mathbf{C}_i\}_{0 \leq i \leq k}$.

In the later sections, we directly call the RGSW extension algorithm with the cGSW encryption for simplicity. It is also worth noting that the cLEV encryptions of $s_{i,j}$ required for the RGSW extension can be seen as a cGSW encryption of 1.

Full Homomorphism. As homomorphic operations incur noise growth, the bootstrapping operation is required to refresh the ciphertext and consequently obtain full homomorphism. Roughly speaking, the bootstrapping takes a ciphertext and the bootstrapping key, and homomorphically computes the decryption circuit. It is known that the bootstrapping for GSW only requires a polynomial overhead (consuming only one level) with the branching program-based approach [14,3,30,27]. We remark that this approach can be adopted to our scheme with each party disclosing encryptions of each bit of their secret key, since essentially the decryption circuit of our scheme can be written in a branching program. Therefore, as long as the ciphertext modulus can tolerate the decryption circuit (with some margin for homomorphic computation and plaintext), our scheme is fully homomorphic⁸.

Role of CRS. While the majority of the existing MKFHE schemes are based on the CRS model, there are a few works that consider multi-key schemes in the plain model. Among them, the construction of [44] stands out with its non-interactive key generation and encryption algorithm. Their construction is based on the NTRU assumption, and later it was extended to the NGS encryption scheme [8], concurrently in [59] and [50] to accelerate the TFHE bootstrapping in the multi-key setting. In these works, the joint key after multiplication has a multiplicative structure, and the noise grows exponentially with respect to the number of parties. In contrast, the multi-key schemes that assume the CRS model have a linear joint key structure after multiplication and also show polynomial noise growth with respect to the number of parties.

We note that our scheme also has a multiplicative joint key structure and exhibits exponential noise growth, as shown in Eq. (1) and Lemma 3. Based on this observation, we introduce a conjecture on the relation between the CRS model and the key structure.

Conjecture 1. The usage of Common Reference/Random String (CRS) in the MKFHE allows us to have a linear joint key structure and a polynomial noise growth.

⁸ It is worth noting that this family of bootstrapping is highly optimized in recent works [21,41]. We provide a more detailed description of generating the bootstrapping keys for these recent schemes in Section 5.2.

3.4 Noise Analysis

In this section, we provide a noise analysis for the vector gadget product and the RGSW extension algorithm. We first begin by specifying the key and noise distributions. In typical settings for FHE, the key distribution is chosen to be a ternary distribution with fixed Hamming weight H and the noise distribution is chosen to be some discrete Gaussian distribution with small width σ . In the following analysis, we follow this tradition and assume that the parties sample their keys and noises from these distributions. We also denote the bound for the gadget decomposition h as B_h , and the probabilistic bound for the noise distribution as B_σ . Finally, we use the same notation as the algorithm descriptions from the previous subsections, for simplicity.

Lemma 2 (Vector Gadget Product). *Given a vector $\mathbf{c} \in R_Q^{1 \times k}$ and a (fresh) cLEV encryption $\mathbf{C} \in R_Q^{dk \times (k+1)}$ of μ under the secret $\mathbf{s}$ for the dimension k , let $\mathbf{c}' \leftarrow \mathbf{c} \boxtimes_v \mathbf{C}$. Then, for the noise vector $\mathbf{e} := \mathbf{c}' \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{s} \end{bmatrix} - \mu \cdot \mathbf{c} \pmod{Q}$, it holds that $\|\mathbf{e}\|_\infty \leq kdB_\sigma B_h N$.*

Proof. See Appendix A.

Proposition 1. *For the joint secret key $\overline{\mathbf{sk}} = (\overline{s}_0 = 1, \overline{s}_1, \dots, \overline{s}_n)$, it holds that $\|\overline{s}_i\|_\infty \leq (H+1)^{i-1}$ for $1 \leq i \leq n$.*

Proof. Due the recurrence relation from Eq. (5), we have the following:

$$\|\overline{s}_i\|_\infty = \left\| \sum_{k=0}^{i-1} s_{i,k} \overline{s}_k \right\|_\infty \leq \sum_{k=0}^{i-1} \|s_{i,k} \overline{s}_k\|_\infty \leq 1 + H \cdot \sum_{k=1}^{i-1} \|\overline{s}_k\|_\infty.$$

With $\|\overline{s}_1\|_\infty = 1$, we can inductively obtain the upper norm bound $(H+1)^{i-1}$ for $\overline{s}_i$. $\square$

Lemma 3 (RGSW Extension). *Given (fresh) cLEV encryptions $\text{cLEV}_{s_i}(s_{i,j})$ for $1 \leq i \leq n$ and $1 \leq j \leq i$, and a (fresh) cLEV encryption $\text{cLEV}_{s_{\text{idx}}}(\mu)$ of the idx -th party, let $\overline{\mathbf{C}}$ be the output of Algorithm 1. Then, for the noise vector $\mathbf{e} = (e_0, \dots, e_{d(n+1)-1}) := \overline{\mathbf{C}} \cdot \overline{\mathbf{sk}} - \mu \cdot \mathbf{g} \otimes \overline{\mathbf{sk}} \pmod{Q}$, the following inequality holds,*

$$\|e_{di+j}\|_\infty \lesssim \begin{cases} B_\sigma & \text{if } i = 0 \\ \frac{1}{3} \|\mu\|_1 dN^2 B_\sigma B_h i^3 (H+1)^{i-2} + \text{idx} dN B_\sigma B_h & \text{if } 1 \leq i < \text{idx} \\ \frac{1}{3} \|\mu\|_1 dN^2 B_\sigma B_h \text{idx}^3 (H+1)^{\text{idx}-2} & \text{if } i = \text{idx} \\ \frac{1}{3} dN^2 B_\sigma B_h (H+1)^{i-2} (i^3 + \|\mu\|_1 (\text{idx}+1)^3 (H+1)) & \text{otherwise} \end{cases}$$

for all $0 \leq i \leq n$ and $0 \leq j < d$.

Proof. See Appendix A. $\square$

4 MKFHE in the plain model

In this section, we formalize our multi-key RGSW scheme and discuss how it can be used as a baseline solution for a two-round multi-party computation (MPC) protocol.

4.1 Building Blocks

In this section, we introduce some important building blocks required to realize the traditional syntax of the multi-key homomorphic encryption scheme with our multi-key RGSW scheme. Specifically, we devise a ciphertext shrinking method for the cLEV encryption, a key-switching key generation algorithm for our RGSW scheme, and finally formalize the evaluation algorithm with our multi-key RGSW scheme.

Ciphertext Shrinking. Note that our multi-key scheme from Section 3.2 requires each party to know the exact order in which its ciphertexts are multiplied beforehand since the sizes of the input cLEV ciphertexts differ with respect to the multiplication order. However, this contradicts the definition of MKFHE, as the key generation algorithm only takes the public parameters, not the index of the party. We can handle this issue by having each party output the cLEV encryptions of the maximum size and shrink the size of the cLEV encryption upon receiving them. Let us elaborate in detail. Recall that the cLEV encryption of $\mu \in R$ under the secret $\mathbf{s} = (s_1, \dots, s_n) \in R^n$ with respect to the gadget vector $\mathbf{g} \in \mathbb{Z}^d$ and the corresponding decomposition function $h : R_Q \rightarrow R^d$ is given by

$\mathbf{C} = (c_{i,j})_{1 \leq i \leq nd, 1 \leq j \leq n+1} \in R_Q^{nd \times (n+1)}$, such that $\mathbf{C} \cdot \begin{bmatrix} \mathbf{I}_n \\ \mathbf{s}^\top \end{bmatrix} \approx \mu \cdot \mathbf{g} \otimes \mathbf{I}_n \pmod{Q}$.

From this relation, we can easily show that

$$\mathbf{C}' = \begin{bmatrix} c_{1,1} & \dots & c_{1,k} & c_{1,n+1} \\ \vdots & \ddots & \vdots & \vdots \\ c_{kd,1} & \dots & c_{kd,k} & c_{kd,n+1} \end{bmatrix} \in R_Q^{kd \times (k+1)}$$

is a cLEV encryption of μ under the secret $\mathbf{s}' = (s_1, \dots, s_k)$ for any $1 \leq k \leq n$, since

$$\mathbf{C}' \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{s}'^\top \end{bmatrix} = \begin{bmatrix} c_{1,1} & \dots & c_{1,k} & c_{1,n+1} \\ \vdots & \ddots & \vdots & \vdots \\ c_{kd,1} & \dots & c_{kd,k} & c_{kd,n+1} \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \\ s_1 & s_2 & \dots & s_k \end{bmatrix} \approx \mu \cdot \mathbf{g} \otimes \mathbf{I}_k \pmod{Q}.$$

We note that the ciphertext shrinking can be directly applied to the cGSW encryption as well.

Key-Switching Key Generation. In real-world protocols using homomorphic encryption schemes with multiple parties, there can be more than two evaluators, where each evaluates its circuit and then potentially evaluates another circuit on their outputs. However, their choices of indices for parties may differ in the asymmetric setting, making the joint secret of each evaluator completely different. We remark that these secrets cannot be interchanged with a simple rearrangement, unlike the prior works in the CRS model. This raises the need for key-switching keys between these ciphertexts.

Suppose that we want to switch the joint secret from $\overline{\mathbf{sk}} = (\overline{s}_0 = 1, \overline{s}_1, \dots, \overline{s}_n)$ to $\overline{\mathbf{sk}}' = (\overline{s}'_0 = 1, \overline{s}'_1, \dots, \overline{s}'_n)$. Then, we require the key-switching key $\mathbf{ksk}_{\overline{\mathbf{sk}} \rightarrow \overline{\mathbf{sk}}'} \in R_Q^{nd \times (n+1)}$ from $\overline{\mathbf{sk}}$ to $\overline{\mathbf{sk}}'$, which satisfies the following relation.

$$\mathbf{ksk}_{\overline{\mathbf{sk}} \rightarrow \overline{\mathbf{sk}}'} \cdot \overline{\mathbf{sk}}' \approx \mathbf{g} \otimes \begin{bmatrix} \overline{s}_1 \\ \vdots \\ \overline{s}_n \end{bmatrix} \pmod{Q}$$

Given an MLWE ciphertext $\mathbf{c} = (c_0, \dots, c_n) \in R_Q^{n+1}$, its key can be switched from $\overline{\mathbf{sk}}$ to $\overline{\mathbf{sk}}'$ as follows:

$$\mathbf{c}' = (c_0, 0, \dots, 0) + [h(c_1)^\top \dots h(c_n)^\top] \cdot \mathbf{ksk}_{\overline{\mathbf{sk}} \rightarrow \overline{\mathbf{sk}}'} \pmod{Q}.$$

It is easy to show that this algorithm is valid since

$$\mathbf{c}' \cdot \overline{\mathbf{sk}}' \approx c_0 + [h(c_1)^\top \dots h(c_n)^\top] \cdot \mathbf{g} \otimes \begin{bmatrix} \overline{s}_1 \\ \vdots \\ \overline{s}_n \end{bmatrix} = c_0 + \sum_{i=1}^n c_i s_i = \mathbf{c} \cdot \overline{\mathbf{sk}} \pmod{Q}.$$

Now, observe that the key-switching key $\mathbf{ksk}_{\overline{\mathbf{sk}} \rightarrow \overline{\mathbf{sk}}'}$ from $\overline{\mathbf{sk}}$ to $\overline{\mathbf{sk}}'$ is essentially a stack of MLWE encryptions of $g_j \overline{s}_i$ for $1 \leq i \leq n, 0 \leq j < d$ under $\overline{\mathbf{sk}}'$. Naïvely, this can be generated from the cLEV encryptions of entries of each party's secret, and $g_j \overline{s}_i$ is a multivariate polynomial of them. A more efficient (and conceptually simpler) way to achieve this is to utilize the internal product.

Roughly speaking, we first generate the RGSW encryptions of the entries of each party's secret under $\overline{\mathbf{sk}}$, and then utilize the internal product to generate MLWE encryptions of $\overline{s}_i$ from them. With this approach, we can use a similar iterative method for RGSW generation. We describe the key-switching key generation algorithm in Algorithm 2. In the algorithm description, $\mathbf{s}_i = (s_{i,0}, \dots, s_{i,n-1}) \in R^n$ denotes the secret of the i -th party. We also consider two arrays idx and idx' of length n that hold the indices corresponding to the secret $\overline{\mathbf{sk}}$ and $\overline{\mathbf{sk}}'$, respectively. In other words,

$$\overline{\mathbf{sk}} = \prod_{i=1}^n \begin{bmatrix} 1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 1 \\ s_{idx[i],1} & \dots & s_{idx[i],idx[i]} \end{bmatrix}, \quad \overline{\mathbf{sk}}' = \prod_{i=1}^n \begin{bmatrix} 1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 1 \\ s_{idx'[i],1} & \dots & s_{idx'[i],idx'[i]} \end{bmatrix}.$$

In Algorithm 2, we describe the key-switching key generation for the multi-key RGSW encryption with different indices of parties.

Algorithm 2 Key-switching Key Generation

Input: cLEV encryptions $\text{cLEV}_{s_i}(s_{i,j})$ for $1 \leq i \leq n, 0 \leq j < n$, two index arrays idx and idx' corresponding to the joint keys $\overline{\text{sk}}$ and $\overline{\text{sk}}'$, respectively.

- 1: **for** $1 \leq i \leq n, 0 \leq j < idx'[i]$ **do**
- 2: Shrink the cLEV encryption $\text{cLEV}_{s_i}(s_{i,j})$ to the dimension $idx'[i]$.
- 3: **end for**
- 4: **for** $1 \leq i \leq n, 0 \leq j < idx[i]$ **do**
- 5: Run Algorithm 1 to obtain the RGSW encryption $\overline{\mathbf{C}}_{i,j}$ of $s_{i,j}$ under secret $\overline{\text{sk}}'$.
- 6: **end for**
- 7: **for** $0 \leq i < d$ **do**
- 8: $\text{acc}_i \leftarrow (g_i, 0, \dots, 0) \in R_Q^{n+1}$
- 9: **end for**
- 10: **for** $1 \leq i \leq n, 0 \leq j < d$ **do**
- 11: $\text{acc}_{di+j} \leftarrow \sum_{k=0}^{i-1} \text{acc}_{dk+j} \boxtimes \overline{\mathbf{C}}_{i,k}$
- 12: **end for**

Output: $\text{ksk}_{\overline{\text{sk}} \rightarrow \overline{\text{sk}}'} = \begin{bmatrix} \text{acc}_d \\ \vdots \\ \text{acc}_{d(n+1)-1} \end{bmatrix}$

Correctness. We demonstrate the correctness of the key-switching key generation algorithm using mathematical induction. Our claim is that acc_{di+j} is an MLWE encryption of $g_j \overline{s}_i$ under the secret key $\overline{\text{sk}}'$. Trivially, acc_j is an MLWE encryption of $g_j \overline{s}_0 = g_j$. Now, assume that the claim holds for all $i \leq \iota$ for some $0 \leq \iota < n$. Then, due to the correctness of the external product, $\text{acc}_{dk+j} \boxtimes \overline{\mathbf{C}}_{\iota+1,k}$ is an MLWE encryption of $g_j \overline{s}_k \cdot s_{\iota+1,k}$ for all $0 \leq k \leq \iota$. Then, $\text{acc}_{d\iota+j}$ decrypts to $g_j \overline{s}_{\iota+1} = \sum_{k=0}^{\iota} g_j \overline{s}_k s_{\iota+1,k}$ due to Eq. (5). Therefore, the claim holds for $\iota + 1$ as well. Consequently, by mathematical induction, acc_{di+j} is an MLWE encryption of $g_j \overline{s}_i$ for all $1 \leq i \leq n$ and $0 \leq j < d$, and therefore, the output is a valid key-switching key from $\overline{\text{sk}}$ to $\overline{\text{sk}}'$. We provide a formal noise analysis in Appendix A.

Remark 1. We note that the input index set idx and output index set idx' can contain different parties. However, it is obvious that idx should be a subset of idx' , and then we can simply append the missing parties to the end of the index set idx and obtain a new index set $\overline{idx}$. Subsequently, as observed in Section 3.2, we can easily convert the ciphertext under the joint key with respect to idx into one under the joint key with respect to $\overline{idx}$ by appending some zeros to the ciphertext. Therefore, we can easily perform key-switching with the key switching key from multiplicative index $\overline{idx}$ to idx' .

Circuit Evaluation. Finally, we describe the circuit evaluation with our multi-key RGSW scheme. Assume a circuit $\mathcal{C}$ that takes the messages $\mu_1, \dots, \mu_m$ as

input. Also, let K be the maximum possible number of parties. Then, each party outputs the cGSW encryption of their messages and the cLEV encryptions of each entry of the secret key, of dimension K . Upon receiving these encryptions, the evaluator first picks the multiplication order. Next, it uses ciphertext shrinking to match the dimension of the input ciphertexts with the multiplication order. Next, it applies the RGSW extension algorithm to the input ciphertexts to generate the RGSW encryptions of $\mu_1, \dots, \mu_m$. Then, the evaluator directly uses the circuit evaluation algorithm of the RGSW scheme.

The input to the circuit may not be a fresh ciphertext but rather outputs of another circuit evaluated by different evaluators. In this case, the evaluators may have chosen different multiplication orders. Then, the key-switching key generation algorithm can be run to generate key-switching keys, and the secrets of the input ciphertexts can be switched to one universal secret. Then, the RGSW circuit evaluation can be applied in a black-box manner again.

4.2 Scheme Description.

Now we formally describe our scheme.

• **MKFHE.Setup**($1^\lambda, 1^K$) : On input a security parameter λ and the maximum number of parties K , choose the ciphertext dimension N , the ciphertext modulus Q , the Hamming weight H of the secret, the noise distribution ψ over $R := \mathbb{Z}[X]/(X^N + 1)$, the gadget vector $\mathbf{g} \in \mathbb{Z}^d$, and the corresponding decomposition function $h : R_Q \rightarrow R^d$. Return the public parameter $\mathbf{pp} = (K, N, Q, H, \psi, \mathbf{g}, h)$.

• **MKFHE.KeyGen**($\mathbf{pp}$) : On input a public parameter $\mathbf{pp}$, sample s_i uniformly from the set of ternary polynomials with Hamming weight H , and set $\mathbf{s} = (s_0, \dots, s_{K-1}) \leftarrow R^K$. Let $\mathbf{ek}_i \leftarrow \text{cLEV.Enc}(\mathbf{s}; s_i)$ for $0 \leq i < K$ and set the evaluation key $\mathbf{ek} = \{\mathbf{ek}_i\}_{0 \leq i < K}$. Sample $a \leftarrow R_Q$ and $e_i \leftarrow \psi$ for $0 \leq i < K$, and set $\mathbf{pk} = (b_0, b_1, \dots, b_{K-1}, a) \in R_Q^{K+1}$ where $b_i = -a \cdot s_i + e_i \pmod{Q}$ for $0 \leq i < K$. Return $(\mathbf{s}, \mathbf{ek}, \mathbf{pk})$.

• **MKFHE.Enc**($\mathbf{s}; \mu$) : On input a secret key $\mathbf{s}$ and a message $\mu \in R$, set $\mathbf{C} \leftarrow \text{cGSW.Encrypt}(\mathbf{s}; \mu)$ and return $\mathbf{ct}^9$.

• **MKFHE.Eval**($\mathcal{C}, \{(\mathbf{ek}_i, \mathbf{pk}_i, \mathbf{ct}_i)\}_{i \in [n]}$) : On input the circuit $\mathcal{C}$ and n tuples $(\mathbf{ek}_i, \mathbf{pk}_i, \mathbf{ct}_i)$ for some $1 \leq n \leq K$ where each consists of an evaluation key $\mathbf{ek}_i$, a public key $\mathbf{pk}_i$, and cGSW ciphertexts $\mathbf{ct}_i$, do as described in Section 4.1. More specifically,

1. Choose a multiplication order as a list $\mathbf{idx} \in \mathbb{Z}^n$.
2. Perform the ciphertext shrinking from Section 4.1 on the cGSW ciphertexts $\{\mathbf{ct}_i\}_{i \in [n]}$ with respect to the multiplication order $\mathbf{idx}$ ¹⁰.

⁹ This encryption algorithm takes the secret key instead of the public key, unlike the original definition of multi-key homomorphic encryption for simplicity. Note that this can be easily adjusted.

¹⁰ There can be several ways to choose the index set. For example, we can choose the order based on the alphabetical order of the ciphertexts to ensure that the order remains the same for all evaluators.

3. Perform the RGSW extension algorithm from Section 3.2 on the shrunk cGSW ciphertexts using the evaluation and public keys $\{\text{ek}_i\}_{i \in [n]}$ and $\{\text{pk}_i\}_{i \in [n]}$ to obtain multi-key RGSW ciphertexts $\{\text{CT}_i\}_{i \in [n]}$, where each ciphertext contains information on the multiplication order idx .
4. Compute the circuit $\mathcal{C}$ from $\{\text{CT}_i\}_{i \in [n]}$ using the internal product algorithm and ciphertext addition.
5. Return the output multi-key RGSW ciphertext CT .

• **MKFHE.Dec**($\{\text{s}_i\}_{i \in [n]}; \text{CT}$) : On input all participants' secret keys and a multi-key RGSW ciphertext CT , do the following with a secure MPC protocol.

1. Compute the joint key

$$\overline{\text{sk}} = \prod_{i=1}^n \begin{bmatrix} 1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 1 \\ s_{idx[i],1} & \dots & s_{idx[i],idx[i]} \end{bmatrix}$$

with respect to the multiplicative order of CT .

2. Output $\text{RGSW.Dec}(\text{sk}; \text{CT})$.

We also show the correctness and compactness and address the security of our MKFHE scheme below.

Correctness. By Lemma 3, each party's cGSW encryptions can be switched to the multi-key RGSW ciphertexts that encrypt the same underlying message. Therefore, due to the correctness of the circuit evaluation from the RGSW scheme, the correctness of our scheme holds.

Compactness. The output RGSW ciphertext from Algorithm 1 has a size of $dn(n+1)N$. In the typical RLWE setting, we set $\log Q, N \in O(\lambda)$ for the security parameter λ . Since $d = O(\log Q)$, our ciphertext has an asymptotic size of $O(n^2\lambda^2)$, which is a bivariate polynomial of n and λ . Since the RGSW scheme preserves the size after the circuit evaluation, our scheme also satisfies the compactness from the definition of MKFHE.

Security. The security of our scheme relies on the shared-A RLWE hardness problem and the circular security assumption.

Achieving Stronger Security. We also discuss how to achieve stronger security for our MKFHE. As discussed in [44,22], a semi-honest MKFHE scheme can be transformed into a semi-malicious scheme by proving the well-formedness of the input encryptions and evaluation keys of each party. These can be achieved with non-interactive zero-knowledge proofs (NIZKs), without any additional communication rounds. It is worth noting that there are some RLWE-specific NIZKs with concrete performance for n -out-of- n threshold FHE schemes [15,31], which can be applied to our scheme as well, thereby achieving stronger security.

4.3 Two-round Semi-Malicious Secure MPC

Finally, we provide a brief overview of how to build a two-round semi-malicious MPC protocol based on the transformation from [4].

One-round decryption. As described in prior works [44,38,39,4], the distributed decryption process can be instantiated with another MPC protocol, including multi-party (threshold) FHE or a garbled circuit. However, the use of a general MPC would substantially increase the round complexity of the decryption protocol. Therefore, we leverage the transformation from Ananth et al. [4] to achieve one-round decryption for our MKFHE scheme. In [4], the authors built an MKFHE scheme in the plain model with one-round decryption from three ingredients, as we describe in Theorem 1.

Theorem 1 ([4, Thm 1.1]). *Assuming,*

1. *Multi-key FHE with trusted setup and one-round decryption,*
2. *Multi-key FHE in the plain model but with arbitrary round decryption,*
3. *Two-round semi-malicious oblivious transfer in the plain model*

there exists multi-key FHE in the plain model with one-round decryption.

Briefly speaking, they evaluate the ‘complex’ decryption circuit of an MKFHE scheme in the plain model with arbitrary round decryption, using a non-compact multi-key homomorphic encryption scheme made from the OT and an MKFHE scheme with trusted setup (this is referred to as MHE in [4]). Therefore, this construction can be naturally extended to our MKFHE scheme by realizing the decryption algorithm of our MKFHE scheme with this MHE. Note that the authors provided a concrete instantiation of MHE solely based on the hardness of the (R)LWE problem, from existing LWE-based MKFHE schemes including [48,51,16,39,17] and LWE-based OT protocols such as [10,7,47,43,60,61]. Consequently, we can build an MKFHE in the plain model by combining these works and our MKFHE, due to Theorem 1. For a more detailed construction, we refer to Section 8 of [4].

Two-round MPC protocol. Based on the one-round decryption, a two-round MPC directly follows from the construction of our MKFHE scheme. We adopt a classical server-(multiple) clients model, where a server computes a function value on behalf of all clients. In the first round, each party outputs the evaluation key, public key, and encryptions with a proof that these materials are well-formed by using a non-interactive zero-knowledge proof (NIZK) system. Then, the server verifies the correctness of the inputs and proceeds to evaluate the circuit if it is correct. Subsequently, in the second round, the evaluation results can be decrypted using the one-round decryption discussed above, with NIZK proving that the decryption share from each party is honestly generated.

For the proof that our two-round MPC is secure against semi-malicious adversaries, we refer to Theorem 8.1 of [4]. Since semi-malicious secure MKFHE implies semi-malicious MPC [48], it suffices to show that our MKFHE with one-round decryption is secure against semi-malicious adversaries. Since our MKFHE

with unstructured decryption Section 4.2 is semantically secure, we employ semi-malicious two-round oblivious transfer in the plain model. Thus, our MKFHE built upon such building blocks is secure against semi-malicious adversaries according to the theorem.

5 Multi-key FHE Key Generation

In this section, we explain how our multi-key RGSW scheme can be used as a baseline for the construction of a multi-key variant of the existing RLWE-based FHE schemes, such as BFV [9,28], CKKS [20], TFHE [21], LMKCDEY [41], and Carousel [33]. The main difficulty in extending a single-key scheme to a multi-key scheme lies in the evaluation key generation. This is mainly because a single-key shared-A ciphertext can be easily converted into a multi-key ciphertext with simple zero padding, as discussed in Section 3.2, whereas the evaluation keys are essentially encryptions of some multivariate polynomial of each party's secret. Therefore, in this section, we only discuss how to generate the evaluation keys for a variety of homomorphic encryption schemes.

5.1 BFV/CKKS

We consider the key generation for multi-key BFV and CKKS scheme¹¹. These two schemes have the same evaluation key structure, although they have different functionalities; one supports finite field arithmetic, whereas the other supports fixed-point arithmetic. Roughly speaking, their evaluation key consists of two types of key-switching keys, namely the automorphism key and the relinearization key. Below, we describe how to generate them one by one.

Automorphism Keys The automorphism key with respect to the automorphism Ψ is given by a key-switching key from $\Psi(\overline{\mathbf{sk}}) = (\Psi(s_0) = 1, \Psi(s_1), \dots, \Psi(s_n))$ to $\overline{\mathbf{sk}}$. Given an MLWE encryption $\mathbf{c} = (c_0, \dots, c_n) \in R_Q^{n+1}$ of the plaintext m under the secret $\overline{\mathbf{sk}}$, i.e., $\mathbf{c} \cdot \overline{\mathbf{sk}} = \sum_{i=0}^n c_i \overline{s}_i \approx m \pmod{Q}$, it is easy to see that $\Psi(\mathbf{c}) = (\Psi(c_0), \dots, \Psi(c_n)) \in R_Q^{n+1}$ is an encryption of $\Psi(m)$ under the secret $\Psi(\overline{\mathbf{sk}})$ since

$$\Psi(\mathbf{c}) \cdot \Psi(\overline{\mathbf{sk}}) = \sum_{i=0}^n \Psi(c_i) \cdot \Psi(\overline{s}_i) \approx \Psi(m) \pmod{Q}.$$

Therefore, we can perform the homomorphic automorphism operation by simply switching the secret key of $\Psi(\mathbf{c})$ from $\Psi(\overline{\mathbf{sk}})$ to $\overline{\mathbf{sk}}$.

To generate the key-switching key, we utilize a similar induction from Algorithm 1. Note that the secret key $\Psi(\overline{\mathbf{sk}})$ can be iteratively generated with the recurrence relation Eq. (5) in the same manner as $\overline{\mathbf{sk}}$. We describe the full algorithm in Algorithm 3.

¹¹ We remark that the key generation for BGV scheme is also feasible. We opt it out from the paper since it has a slightly different key structure compared to BFV and CKKS.

Algorithm 3 Automorphism Key generation

Input: cLEV encryptions $\text{cLEV}_{\mathbf{s}_i}(\Psi(s_{i,j}))$ for $1 \leq i \leq n$ and $1 \leq j \leq i$.

```

1: for  $0 \leq i < d$  do
2:    $\text{acc}_i^0 \leftarrow (g_i)$ 
3: end for
4: for  $1 \leq i \leq n, 0 \leq j < d$  do
5:   for  $0 \leq k < i$  do
6:      $\text{acc}_{dk+j}^i \leftarrow (\text{acc}_{dk+j}^{i-1} \mid 0)$ 
7:   end for
8:    $\text{acc}_{di+j}^i \leftarrow \sum_{k=0}^{i-1} \text{acc}_{dk+j}^{i-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_i}(\Psi(s_{i,j}))$ 
9: end for

```

Output: $\text{ksk}_{\psi(\overline{\mathbf{s}}k) \rightarrow \overline{\mathbf{s}}k} = \begin{bmatrix} \text{acc}_d^n \\ \vdots \\ \text{acc}_{d(n+1)-1}^n \end{bmatrix}$

Relinearization Key The relinearization key is a special form of key-switching key that is necessary to perform homomorphic multiplication with the BFV or CKKS scheme. Let us briefly explain. Suppose that we are given two MLWE encryptions $\mathbf{c} = (c_0, \dots, c_n) \in R_Q^{n+1}$ and $\mathbf{c}' = (c'_0, \dots, c'_n) \in R_Q^{n+1}$ of plaintexts m and m' under the secret $\overline{\mathbf{s}}k$, i.e., $\mathbf{c} \cdot \overline{\mathbf{s}}k = \sum_{i=0}^n c_i \overline{s}_i \approx m \pmod{Q}$ and $\mathbf{c}' \cdot \overline{\mathbf{s}}k = \sum_{i=0}^n c'_i \overline{s}_i \approx m' \pmod{Q}$. Then, it is easy to see that the tensor $\mathbf{c} \otimes \mathbf{c}'$ of $\mathbf{c}$ and $\mathbf{c}'$ is an MLWE encryption of $m \cdot m'$ under the tensored key $\overline{\mathbf{s}}k \otimes \overline{\mathbf{s}}k$, since

$$(\mathbf{c} \otimes \mathbf{c}') \cdot (\overline{\mathbf{s}}k \otimes \overline{\mathbf{s}}k) = (\mathbf{c} \cdot \overline{\mathbf{s}}k) \cdot (\mathbf{c}' \cdot \overline{\mathbf{s}}k) \approx m \cdot m' \pmod{Q}^{12}.$$

Therefore, the homomorphic multiplication can be performed by switching the secret of the ciphertext $\mathbf{c} \otimes \mathbf{c}'$ from the tensored key $\overline{\mathbf{s}}k \otimes \overline{\mathbf{s}}k$ to the original key $\overline{\mathbf{s}}k$. In the prior multi-key schemes [17,37], this key-switching was done with n external products, given the RGSW encryptions of $\overline{s}_1, \dots, \overline{s}_n$. Specifically, for two input ciphertexts $\mathbf{c} = (c_0, \dots, c_n) \in R_Q^{n+1}$ and $\mathbf{c}' = (c'_0, \dots, c'_n) \in R_Q^{n+1}$, (roughly) we compute the following¹³.

$$c_0 \cdot \mathbf{c}' + \sum_{i=1}^n (c_i \cdot \mathbf{c}') \boxtimes \text{RGSW}(\overline{s}_i) \pmod{Q}.$$

Following this approach, we only need to generate the encryptions of RGSW encryptions of $\overline{s}_i, 1 \leq i \leq n$. To achieve this, we first generate RGSW encryptions of $s_{i,j}$ using Algorithm 1. Then, we homomorphically compute Eq. (5) with the RGSW cryptosystem, in order to obtain the RGSW encryptions of $\overline{s}_i$ iteratively. We describe the full algorithm in Algorithm 4.

¹² We remark that the BFV scheme has a different tensoring process due to its MSB encoding strategy. Nevertheless, the structure of the relinearization key remains the same.

¹³ The performance can be improved with re-ordering of entries, however, the asymptotic complexity remains the same.

Algorithm 4 Relinearization Key Generation

Input: cLEV encryptions $\text{cLEV}_{s_i}(s_{i,j})$ for $1 \leq i \leq n$ and $0 \leq j < i$.

1: **for** $1 \leq i \leq n, 0 \leq j < i$ **do**

2: Run Algorithm 1 to obtain the RGSW encryption $\overline{\mathbf{C}}_{i,j}$ of $s_{i,j}$.

3: **end for**

4: **for** $1 \leq i \leq n$ **do**

5: $\text{rlk}_i \leftarrow \overline{\mathbf{C}}_{i,0} + \sum_{k=1}^{i-1} \text{rlk}_k \boxtimes \overline{\mathbf{C}}_{i,k}$

6: **end for**

Output: $\{\text{rlk}_i\}_{1 \leq i \leq n}$

5.2 TFHE/LMKCDEY/Carousel

The homomorphic encryption schemes such as TFHE, LMKCDEY, and Carousel (often called as AP[3] family) require a special bootstrapping key, called the blind rotation key, on top of the optional automorphism keys. Suppose that each coefficient $\bar{s}_{i,j}$ of the secret $\bar{\mathbf{s}}\mathbf{k}$ is confined in some interval $[-B, B]$ for some integer B , for $1 \leq i \leq n$ and $0 \leq j < N$. Then, TFHE and Carousel require the RGSW encryptions of a binary value that indicates if $\bar{s}_{i,j} = k$ or not, for all $-B \leq k \leq B$. On the other hand, LMKCDEY scheme requires the RGSW encryptions of $X^{\bar{s}_{i,j}}$.

We remark that the blind rotation key can be easily obtained with a homomorphic evaluation of Eq. (5), but with a coefficient-to-coefficient naïve multiplication. Firstly, we describe the generation of the blind rotation key for TFHE/Carousel. Assume that we are given two numbers $-B_x \leq x \leq B_x, -B_y \leq y \leq B_y$ for bounds $B_x, B_y > 0$, given in the indicator form

$$x_i = \begin{cases} 1 & \text{if } x = i \\ 0 & \text{otherwise} \end{cases} \quad \text{for } -B_x \leq i \leq B_x,$$

$$y_i = \begin{cases} 1 & \text{if } y = i \\ 0 & \text{otherwise} \end{cases} \quad \text{for } -B_y \leq i \leq B_y.$$

Then, we can obtain the indicator form of $z^+ = x + y$ and $z^\times = xy$ as follows:

$$z_i^+ = \sum_{\substack{-B_x \leq j \leq B_x, \\ -B_y \leq k \leq B_y, \\ j+k=i}} x_i \cdot y_i \quad \text{for } -B_x - B_y \leq i \leq B_x + B_y, \quad (6)$$

$$z_i^\times = \sum_{\substack{-B_x \leq j \leq B_x, \\ -B_y \leq k \leq B_y, \\ j \cdot k = i}} x_i \cdot y_i \quad \text{for } -B_x B_y \leq i \leq B_x B_y. \quad (7)$$

Therefore, we can iteratively compute the indicator form of each coefficient of $\bar{s}_i$'s, given the indicator form of each coefficient of $s_{i,j}$ for all $1 \leq i \leq n$ and $0 \leq j < i$ ¹⁴. The generation of the blind rotation key for LMKCDEY

¹⁴ We remark that this approach can be seen as a generalization of the n -out-of- n threshold TFHE construction by Park and Rovira [49]. In their work, they only considered the addition between the indicator form.

can be directly followed from this, since there is a trivial transformation from the indicator form to the exponent form. To be specific, for the indicator form $y_{-B}, \dots, y_B$ of $-B \leq y \leq B$ for some bound B , we have the following equality:

$$X^y = \sum_{i=-B}^B y_i \cdot X^i.$$

So, we can simply run the blind rotation key generation algorithm for the TFHE/Carousel scheme, and then homomorphically evaluate the equation above, to obtain the blind rotation key for the LMKCDEY scheme. We provide the full algorithm for TFHE/Carousel blind rotation key algorithm in Algorithm 5.

Algorithm 5 Blind Rotation Key Generation

Input: cLEV encryptions of the indicator form of each coefficient of $s_{i,j}$ for $1 \leq i \leq n$ and $0 \leq j < i$.

- 1: Run Algorithm 1 to obtain the RGSW encryptions of the indicator form of each coefficient of $s_{i,j}$ for all $1 \leq i \leq n$ and $0 \leq j < i$.
- 2: **for** $1 \leq i \leq n$ **do**
- 3: Use Eq. (6) and Eq. (7) to iteratively obtain the RGSW encryptions of the indicator form of each coefficient of $\overline{\mathbf{sk}}_i$, using Eq. (4).
- 4: **end for**

Output: Output the RGSW encryptions of the indicator form of each coefficient of $\overline{\mathbf{sk}}_i$ for $1 \leq i \leq n$.

6 Implementation

To show the correctness and to demonstrate the concrete performance of our MKFHE scheme, we provide a proof-of-concept implementation of it. We implemented our scheme on top of HIENAA.jl [32] library, written in Julia. The source code is available at <https://github.com/SNUCP/crsless>.

Our recommended parameter sets allow the parties to evaluate circuits with a depth of at most two. We fix the Hamming weight of the secret keys by $H = 192$, and choose the ring degree N and the bit-length of modulus $\log Q$ to achieve 128-bit security. The lattice-estimator [2] was used to measure the security of our parameter sets. In the experiments, we used the Residue Number System (RNS) gadget decomposition technique. Roughly speaking, the ciphertext modulus Q is a product of d primes $Q_1, \dots, Q_d$ of the size $\log Q/d$, and the gadget decomposition function outputs the ring element modulo Q_i for all $1 \leq i \leq d$. Therefore, B_h naturally becomes $\max(Q_1, \dots, Q_d) \approx Q^{1/d}$. We choose the discrete Gaussian distribution with $\sigma = 3.2$ as the noise distribution, which has the probabilistic bound of $3.2 \times 13.11 \approx 42$ with an overwhelming probability of $1 - 2^{-128}$. By substituting these values into the noise formula for RGSW generation Lemma 3, we can guarantee that the decryption failure probability with our parameters is smaller than 2^{-128} .

K	N	$\log Q$	d
2	2^{12}	100	6
4	2^{13}	214	5
8	2^{13}	214	6

(a)

K	Vector Gadget Product	RGSW Extension	Internal Product	Ciphertext Size	Public Key Size
2	13ms	185ms	243ms	21.3MB → 10.6MB	14.2MB
4	77ms	1159ms	985ms	164.0MB → 41.0MB	131.2MB
8	491ms	12729ms	6964ms	1.530GB → 191.28MB	1.360GB

(b)

Fig. 1. (a) Recommended parameter sets for our MKFHE scheme. K denotes the maximum number of the participants, N denotes the ring dimension, $\log Q$ denotes the bit-length of the ciphertext modulus, and d denotes the gadget length. (b) Performance of vector gadget product, RGSW extension, and internal product algorithms, respectively, and the size of the ciphertexts and public key, with respect to the number of the parties K . In the ciphertext size column, it shows how the size of the single-key cGSW ciphertext changes as it is extended to the multi-key RGSW ciphertext.

We present the recommended parameter sets for our MKFHE in Fig. 1a, and provide the performance of our algorithms under these parameters. All experiments were performed on a machine equipped with an Intel(R) Xeon(R) Platinum 8268 CPU @ 2.90GHz and 376GB RAM, running Ubuntu 20.04.4 LTS. Note that the runtime of the internal product is independent of the underlying multi-key scheme, but purely depends on the dimension of the ciphertext. It is also worth noting that the ciphertext size is reduced after the RGSW extension, as cGSW ciphertext is composed of $O(dK^3)$ ring elements, while RGSW ciphertext is composed of only $O(dK^2)$ ring elements.

References

1. Albrecht, M.R., Bai, S., Ducas, L.: A subfield lattice attack on overstretched NTRU assumptions - cryptanalysis of some FHE and graded encoding schemes. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016, Part I. LNCS, vol. 9814, pp. 153–178. Springer, Berlin, Heidelberg (Aug 2016). https://doi.org/10.1007/978-3-662-53018-4_6
2. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. Journal of Mathematical Cryptology **9**(3), 169–203 (2015)
3. Alperin-Sheriff, J., Peikert, C.: Faster bootstrapping with polynomial error. In: Garay, J.A., Gennaro, R. (eds.) CRYPTO 2014, Part I. LNCS, vol. 8616, pp. 297–314. Springer, Berlin, Heidelberg (Aug 2014). https://doi.org/10.1007/978-3-662-44371-2_17
4. Ananth, P., Jain, A., Jin, Z., Malavolta, G.: Multi-key fully-homomorphic encryption in the plain model. In: Pass, R., Pietrzak, K. (eds.) TCC 2020, Part I. LNCS, vol. 12550, pp. 28–57. Springer, Cham (Nov 2020). https://doi.org/10.1007/978-3-030-64375-1_2

5. Bae, Y., Cheon, J.H., Hanrot, G., Park, J.H., Stehlé, D.: Plaintext-ciphertext matrix multiplication and FHE bootstrapping: Fast and fused. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part III. LNCS, vol. 14922, pp. 387–421. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68382-4_12
6. Bergerat, L., Bonte, C., Curtis, B.R., Orfila, J.B., Paillier, P., Tap, S.: Sharing the mask: Ttfe bootstrapping on packed messages. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2025**(4), 925–971 (2025)
7. Blazy, O., Chevalier, C., Vu, Q.H.: Post-quantum uc-secure oblivious transfer in the standard model with adaptive corruptions. In: Proceedings of the 14th International Conference on Availability, Reliability and Security. ARES '19, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3339252.3339280>, <https://doi.org/10.1145/3339252.3339280>
8. Bonte, C., Iliashenko, I., Park, J., Pereira, H.V.L., Smart, N.P.: FINAL: Faster FHE instantiated with NTRU and LWE. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part II. LNCS, vol. 13792, pp. 188–215. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22966-4_7
9. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 868–886. Springer, Berlin, Heidelberg (Aug 2012). https://doi.org/10.1007/978-3-642-32009-5_50
10. Brakerski, Z., Döttling, N.: Two-message statistically sender-private OT from LWE. In: Beimel, A., Dziembowski, S. (eds.) TCC 2018, Part II. LNCS, vol. 11240, pp. 370–390. Springer, Cham (Nov 2018). https://doi.org/10.1007/978-3-030-03810-6_14
11. Brakerski, Z., Halevi, S., Polychroniadou, A.: Four round secure computation without setup. In: Kalai, Y., Reyzin, L. (eds.) TCC 2017, Part I. LNCS, vol. 10677, pp. 645–677. Springer, Cham (Nov 2017). https://doi.org/10.1007/978-3-319-70500-2_22
12. Brakerski, Z., Langlois, A., Peikert, C., Regev, O., Stehlé, D.: Classical hardness of learning with errors. In: Boneh, D., Roughgarden, T., Feigenbaum, J. (eds.) 45th ACM STOC. pp. 575–584. ACM Press (Jun 2013). <https://doi.org/10.1145/2488608.2488680>
13. Brakerski, Z., Perlman, R.: Lattice-based fully dynamic multi-key FHE with short ciphertexts. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016, Part I. LNCS, vol. 9814, pp. 190–213. Springer, Berlin, Heidelberg (Aug 2016). https://doi.org/10.1007/978-3-662-53018-4_8
14. Brakerski, Z., Vaikuntanathan, V.: Efficient fully homomorphic encryption from (standard) lwe. *SIAM Journal on computing* **43**(2), 831–871 (2014)
15. Chatel, S., Mouchet, C., Sahin, A.U., Pyrgelis, A., Troncoso, C., Hubaux, J.P.: PELTA - shielding multiparty-FHE against malicious adversaries. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023. pp. 711–725. ACM Press (Nov 2023). <https://doi.org/10.1145/3576915.3623139>
16. Chen, H., Chillotti, I., Song, Y.: Multi-key homomorphic encryption from TFHE. In: Galbraith, S.D., Moriai, S. (eds.) ASIACRYPT 2019, Part II. LNCS, vol. 11922, pp. 446–472. Springer, Cham (Dec 2019). https://doi.org/10.1007/978-3-030-34621-8_16
17. Chen, H., Dai, W., Kim, M., Song, Y.: Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) ACM CCS 2019. pp. 395–412. ACM Press (Nov 2019). <https://doi.org/10.1145/3319535.3363207>

18. Chen, L., Zhang, Z., Wang, X.: Batched multi-hop multi-key FHE from ring-LWE with compact ciphertext extension. In: Kalai, Y., Reyzin, L. (eds.) TCC 2017, Part II. LNCS, vol. 10678, pp. 597–627. Springer, Cham (Nov 2017). https://doi.org/10.1007/978-3-319-70503-3_20
19. Cheon, J.H., Jeong, J., Lee, C.: An algorithm for ntru problems and cryptanalysis of the ggh multilinear map without a low-level encoding of zero. LMS Journal of Computation and Mathematics **19**(A), 255–266 (2016). <https://doi.org/10.1112/S1461157016000371>
20. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017, Part I. LNCS, vol. 10624, pp. 409–437. Springer, Cham (Dec 2017). https://doi.org/10.1007/978-3-319-70694-8_15
21. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: Cheon, J.H., Takagi, T. (eds.) ASIACRYPT 2016, Part I. LNCS, vol. 10031, pp. 3–33. Springer, Berlin, Heidelberg (Dec 2016). https://doi.org/10.1007/978-3-662-53887-6_1
22. Chongchitmate, W., Ostrovsky, R.: Circuit-private multi-key FHE. In: Fehr, S. (ed.) PKC 2017, Part II. LNCS, vol. 10175, pp. 241–270. Springer, Berlin, Heidelberg (Mar 2017). https://doi.org/10.1007/978-3-662-54388-7_9
23. Clear, M., McGoldrick, C.: Multi-identity and multi-key leveled FHE from learning with errors. In: Gennaro, R., Robshaw, M.J.B. (eds.) CRYPTO 2015, Part II. LNCS, vol. 9216, pp. 630–656. Springer, Berlin, Heidelberg (Aug 2015). https://doi.org/10.1007/978-3-662-48000-7_31
24. Colombo, L., Falcetta, A., Roveri, M.: Training encrypted neural networks on encrypted data with fully homomorphic encryption. In: Proceedings of the 12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography. p. 64–75. WAHC '24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3689945.3694802>, <https://doi.org/10.1145/3689945.3694802>
25. Cong, K., Das, D., Park, J., Pereira, H.V.L.: SortingHat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022. pp. 563–577. ACM Press (Nov 2022). <https://doi.org/10.1145/3548606.3560702>
26. Cong, K., Kang, J., Nicolas, G., Park, J.: Faster private decision tree evaluation for batched input from homomorphic encryption. In: Galdi, C., Phan, D.H. (eds.) SCN 24, Part II. LNCS, vol. 14974, pp. 3–23. Springer, Cham (Sep 2024). https://doi.org/10.1007/978-3-031-71073-5_1
27. Ducas, L., Micciancio, D.: FHEW: Bootstrapping homomorphic encryption in less than a second. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015, Part I. LNCS, vol. 9056, pp. 617–640. Springer, Berlin, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46800-5_24
28. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144 (2012), <https://eprint.iacr.org/2012/144>
29. Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013, Part I. LNCS, vol. 8042, pp. 75–92. Springer, Berlin, Heidelberg (Aug 2013). https://doi.org/10.1007/978-3-642-40041-4_5
30. Hiromasa, R., Abe, M., Okamoto, T.: Packing messages and optimizing bootstrapping in GSW-FHE. In: Katz, J. (ed.) PKC 2015. LNCS, vol. 9020, pp. 699–

715. Springer, Berlin, Heidelberg (Mar / Apr 2015). https://doi.org/10.1007/978-3-662-46447-2_31
31. Hwang, I., Lee, H., Seo, J., Song, Y.: Practical zero-knowledge PIOP for maliciously secure multiparty homomorphic encryption. In: ACM CCS 2025. pp. 4049–4063. ACM Press (Nov 2025). <https://doi.org/10.1145/3719027.3765229>
32. Hwang, I., Min, S.: HIENAA.jl. Online: <https://github.com/snu-lukemin/HIENAA.jl> (2025)
33. Hwang, I., Min, S., Song, Y.: Carousel: Fully homomorphic encryption with bootstrapping over automorphism group. In: ASIACRYPT 2025, Part VII. pp. 131–164. LNCS, Springer, Singapore (Dec 2025). https://doi.org/10.1007/978-981-95-5122-4_5
34. Ju, J.H., Park, J., Kim, J., Kang, M., Kim, D., Cheon, J.H., Ahn, J.H.: NeuJeans: Private neural network inference with joint optimization of convolution and FHE bootstrapping. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024. pp. 4361–4375. ACM Press (Oct 2024). <https://doi.org/10.1145/3658644.3690375>
35. Kim, E., Lee, H.S., Park, J.: Towards round-optimal secure multiparty computations: Multikey FHE without a CRS. In: Susilo, W., Yang, G. (eds.) ACISP 18. LNCS, vol. 10946, pp. 101–113. Springer, Cham (Jul 2018). https://doi.org/10.1007/978-3-319-93638-3_7
36. Kim, J., Lee, C.: A polynomial time algorithm for breaking NTRU encryption with multiple keys. DCC **91**(8), 2779–2789 (2023). <https://doi.org/10.1007/s10623-023-01233-5>
37. Kim, T., Kwak, H., Lee, D., Seo, J., Song, Y.: Asymptotically faster multi-key homomorphic encryption from homomorphic gadget decomposition. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023. pp. 726–740. ACM Press (Nov 2023). <https://doi.org/10.1145/3576915.3623176>
38. Kraitsberg, M., Lindell, Y., Osheter, V., Smart, N.P., Talibi Alaoui, Y.: Adding distributed decryption and key generation to a ring-LWE based CCA encryption scheme. In: Jang-Jaccard, J., Guo, F. (eds.) ACISP 19. LNCS, vol. 11547, pp. 192–210. Springer, Cham (Jul 2019). https://doi.org/10.1007/978-3-030-21548-4_11
39. Kwak, H., Min, S., Song, Y.: Towards practical multi-key TFHE: Parallelizable, key-compatible, quasi-linear complexity. In: Tang, Q., Teague, V. (eds.) PKC 2024, Part II. LNCS, vol. 14604, pp. 354–385. Springer, Cham (Apr 2024). https://doi.org/10.1007/978-3-031-57728-4_12
40. Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. DCC **75**(3), 565–599 (2015). <https://doi.org/10.1007/s10623-014-9938-4>
41. Lee, Y., Micciancio, D., Kim, A., Choi, R., Deryabin, M., Eom, J., Yoo, D.: Efficient FHEW bootstrapping with small evaluation keys, and applications to threshold homomorphic encryption. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part III. LNCS, vol. 14006, pp. 227–256. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30620-4_8
42. Li, N., Zhou, T., Yang, X., Han, Y., Liu, W., Tu, G.: Efficient multi-key fhe with short extended ciphertexts and directed decryption protocol. IEEE Access **7**, 56724–56732 (2019)
43. Liu, M., Hu, Y.: Universally composable oblivious transfer from ideal lattice. Frontiers of Computer Science **13**(4), 879–906 (2019)
44. López-Alt, A., Tromer, E., Vaikuntanathan, V.: On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In: Karloff, H.J., Pitassi,

- T. (eds.) 44th ACM STOC. pp. 1219–1234. ACM Press (May 2012). <https://doi.org/10.1145/2213977.2214086>
45. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Gilbert, H. (ed.) EUROCRYPT 2010. LNCS, vol. 6110, pp. 1–23. Springer, Berlin, Heidelberg (May / Jun 2010). https://doi.org/10.1007/978-3-642-13190-5_1
 46. Mahdavi, R.A., Ni, H., Linkov, D., Kerschbaum, F.: Level up: Private non-interactive decision tree evaluation using levelled homomorphic encryption. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023. pp. 2945–2958. ACM Press (Nov 2023). <https://doi.org/10.1145/3576915.3623095>
 47. Micciancio, D., Sorrell, J.: Simpler statistically sender private oblivious transfer from ideals of cyclotomic integers. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 381–407. Springer (2020)
 48. Mukherjee, P., Wichs, D.: Two round multiparty computation via multi-key FHE. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part II. LNCS, vol. 9666, pp. 735–763. Springer, Berlin, Heidelberg (May 2016). https://doi.org/10.1007/978-3-662-49896-5_26
 49. Park, J., Rovira, S.: Efficient tfhe bootstrapping in the multiparty setting. *Ieee Access* **11**, 118625–118638 (2023)
 50. Park, J., Van Leeuwen, B., Zajonc, O.: FINALLY: A multi-key FHE scheme based on NTRU and LWE. *CiC* **1**(3), 15 (2024). <https://doi.org/10.62056/aebn-4c2h>
 51. Peikert, C., Shiehian, S.: Multi-key FHE from LWE, revisited. In: Hirt, M., Smith, A.D. (eds.) TCC 2016-B, Part II. LNCS, vol. 9986, pp. 217–238. Springer, Berlin, Heidelberg (Oct / Nov 2016). https://doi.org/10.1007/978-3-662-53644-5_9
 52. Peikert, C., Waters, B.: Lossy trapdoor functions and their applications. In: Ladner, R.E., Dwork, C. (eds.) 40th ACM STOC. pp. 187–196. ACM Press (May 2008). <https://doi.org/10.1145/1374376.1374406>
 53. Research, A.M.L.: Combining machine learning and homomorphic encryption in the apple ecosystem (2024), <https://machinelearning.apple.com/research/homomorphic-encryption>
 54. Solomon, R., Weber, R., Almashaqbeh, G.: smartFHE: Privacy-preserving smart contracts from fully homomorphic encryption. In: 2023 IEEE European Symposium on Security and Privacy. pp. 309–331. IEEE Computer Society Press (Jul 2023). <https://doi.org/10.1109/EuroSP57164.2023.00027>
 55. of Standards, N.I., (NIST), T.: Multi-party threshold cryptography (2025), <https://csrc.nist.gov/projects/threshold-cryptography>
 56. Stehlé, D., Steinfeld, R., Tanaka, K., Xagawa, K.: Efficient public key encryption based on ideal lattices. In: Matsui, M. (ed.) ASIACRYPT 2009. LNCS, vol. 5912, pp. 617–635. Springer, Berlin, Heidelberg (Dec 2009). https://doi.org/10.1007/978-3-642-10366-7_36
 57. Xiang, B., Zhang, J., Wang, K., Deng, Y., Feng, D.: NTRU-based bootstrapping for MK-FHEs without using overstretched parameters. In: Chung, K.M., Sasaki, Y. (eds.) ASIACRYPT 2024, Part I. LNCS, vol. 15484, pp. 241–270. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0875-1_8
 58. Xu, K., Huang, R.: Accelerated multi-key homomorphic encryption via automorphism-based circuit bootstrapping. *IEEE Access* (2024)
 59. Xu, K., Tan, B.H.M., Wang, L.P., Aung, K.M.M., Wang, H.: Multi-key fully homomorphic encryption from ntru and (r)lwe with faster bootstrapping. *Theoretical Computer Science* **968**, 114026 (2023). <https://doi.org/https://doi.org/10.1016/j.tcs.2023.114026>, <https://www.sciencedirect.com/science/article/pii/S0304397523003390>

- 60. Yadav, V.K., Verma, S., Venkatesan, S.: Efficient and secure location-based services scheme in vanet. *IEEE Transactions on Vehicular Technology* **69**(11), 13567–13578 (2020)
- 61. Yadav, V.K., Verma, S., Venkatesan, S.: Linkable privacy-preserving scheme for location-based services. *IEEE Transactions on Intelligent Transportation Systems* **23**(7), 7998–8012 (2021)
- 62. ZAMA: Zama confidential blockchain protocol litepaper (2025), <https://docs.zama.ai/protocol/zama-protocol-litepaper>

A Proofs

A.1 Proof of Lemma 1

Proof. Now we show the correctness of our RGSW extension algorithm from Algorithm 1, using mathematical induction. In the following, we let $\mathbf{sk}_0 = (1)$ for convenience. Firstly, we claim that acc_{dk+j}^i is an encryption of $\bar{s}_k g_j$ if $i < \text{idx}$ and an encryption of $\mu \bar{s}_k g_j$ if $i \geq \text{idx}$, under the secret $\bar{\mathbf{sk}}_i$ for any $0 \leq j < d$ and $0 \leq k \leq i$. Let us first prove for the cases where $i < \text{idx}$. This is trivially true for $i = 0$, since $\bar{s}_0 = 1$ and $\text{acc}_j^0 = (g_j)$ (which trivially decrypts to g_j) for $0 \leq j < d$ by the definition. Now, assume that the claim holds for some $i = \iota$ where $0 \leq \iota < \text{idx} - 1$, i.e., acc_{dk+j}^ι is an MLWE encryption of $\bar{s}_k g_j$ under the secret $\bar{\mathbf{sk}}_\iota$ for all $0 \leq k < \iota$. Then, in the $\iota + 1$, j -th iterations of the outer loop, we do the following two things:

1. $\text{acc}_{dk+j}^{\iota+1} \leftarrow (\text{acc}_{dk+j}^\iota \parallel 0)$ for $0 \leq k \leq \iota$
2. $\text{acc}_{d(\iota+1)+j}^{\iota+1} \leftarrow \sum_{k=0}^\iota \text{acc}_{dk+j}^\iota \boxtimes_v \text{cLEV}_{\mathbf{s}_{\iota+1}}(s_{\iota+1,k})$

Thanks to the recurrence relation on the secret keys $\bar{\mathbf{sk}}_\iota$ and $\bar{\mathbf{sk}}_{\iota+1}$ from Eq. (4), we have

$$\begin{aligned} \text{acc}_{dk+j}^{\iota+1} \cdot \bar{\mathbf{sk}}_{\iota+1} &= [\text{acc}_{dk+j}^\iota \parallel 0] \cdot \bar{\mathbf{sk}}_{\iota+1} \\ &= [\text{acc}_{dk+j}^\iota \parallel 0] \cdot \begin{bmatrix} \bar{\mathbf{sk}}_\iota \\ \mathbf{s}_{\iota+1}^\top \cdot \bar{\mathbf{sk}}_\iota \end{bmatrix} \\ &= \text{acc}_{dk+j}^\iota \cdot \bar{\mathbf{sk}}_\iota \approx \bar{s}_k g_j \pmod{Q} \end{aligned} \quad (8)$$

for any $0 \leq j < d$ and $0 \leq k \leq \iota$. On the other hand, we also have

$$\begin{aligned} \text{acc}_{d(\iota+1)+j}^{\iota+1} \cdot \bar{\mathbf{sk}}_{\iota+1} &= \sum_{k=0}^\iota \text{acc}_{dk+j}^\iota \boxtimes_v \text{cLEV}_{\mathbf{s}_{\iota+1}}(s_{\iota+1,k}) \cdot \bar{\mathbf{sk}}_{\iota+1} \\ &\approx \sum_{k=0}^\iota s_{\iota+1,k} \cdot \text{acc}_{dk+j}^\iota \cdot \bar{\mathbf{sk}}_\iota \\ &\approx \sum_{k=0}^\iota s_{\iota+1,k} \cdot \bar{s}_k g_j = \bar{s}_{\iota+1} g_j \pmod{Q}. \end{aligned} \quad (9)$$

Here, the second line follows from Eq. (1) and Eq. (2), and the final equality follows from Eq. (5). Now, Eq. (8) and Eq. (9) directly imply that the claim is true for $i = \iota + 1$ too. Consequently, by mathematical induction, we can deduce that acc_{dk+j}^i is an encryption of $\bar{s}_k g_j$ for all $0 \leq i < \text{idx}$, $0 \leq j < d$, and $0 \leq k \leq i$. Next, if $i = \text{idx}$, we compute the following three ciphertexts inside the outer loop:

1. $\text{acc}_j^{\text{idx}} \leftarrow \text{cLEV}_{\mathbf{s}_{\text{idx}}}(\mu)_{j+1}$
2. $\text{acc}_{dk+j}^{\text{idx}} \leftarrow \text{acc}_{dk+j}^{\text{idx}-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_{\text{idx}}}(\mu)$ for $0 < k < \text{idx}$
3. $\text{acc}_{d\text{idx}+j}^{\text{idx}} \leftarrow \sum_{k=0}^{\text{idx}-1} \text{acc}_{dk+j}^{\text{idx}-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_{\text{idx}}}(\mu s_{\text{idx},k})$.

From Eq. (2) and Eq. (3), we have the following.

$$\begin{aligned} \text{acc}_j^{\text{idx}} \cdot \overline{\text{sk}}_i &= \text{cLEV}_{\mathbf{s}_{\text{idx}}}(\mu)_{j+1} \cdot \overline{\text{sk}}_i = \text{cLEV}_{\mathbf{s}_{\text{idx}}}(\mu)_{j+1} \cdot \begin{bmatrix} \mathbf{s}_{\text{idx}} \\ \mathbf{I}_{\text{idx}} \end{bmatrix} \cdot \overline{\text{sk}}_{i-1} \\ &\approx [\mu g_j \ 0 \ \dots \ 0] \cdot \overline{\text{sk}}_{i-1} = \mu g_j \pmod{Q} \end{aligned} \quad (10)$$

Also, the correctness of the cLEV encryptions and the previously proven cases where $i < \text{idx}$, we obtain the following two equalities:

$$\begin{aligned} \text{acc}_{dk+j}^{\text{idx}} \cdot \overline{\text{sk}}_{\text{idx}} &= (\text{acc}_{dk+j}^{\text{idx}} \boxtimes_v \text{cLEV}_{\mathbf{s}_{\text{idx}}}(\mu)) \cdot \overline{\text{sk}}_{\text{idx}} \\ &\approx \mu \cdot \text{acc}_{dk+j}^{\text{idx}} \cdot \overline{\text{sk}}_{\text{idx}-1} \approx \mu \overline{s}_k g_j \pmod{Q} \end{aligned} \quad (11)$$

for $0 < k < \text{idx}$, and

$$\begin{aligned} \text{acc}_{d\text{idx}+j}^{\text{idx}} \cdot \overline{\text{sk}}_{\text{idx}} &= \sum_{k=0}^{\text{idx}-1} \left(\text{acc}_{dk+j}^{\text{idx}-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_{\text{idx}}}(\mu s_{\text{idx},k}) \right) \cdot \overline{\text{sk}}_{\text{idx}} \\ &\approx \sum_{k=0}^{\text{idx}-1} \overline{s}_k g_j \cdot \mu s_{\text{idx},k} = \mu \overline{s}_{\text{idx}} g_j \pmod{Q}. \end{aligned} \quad (12)$$

Eq. (10), Eq. (11), and Eq. (12) imply that the claim is true for idx , i.e., $\text{acc}_{dk+j}^{\text{idx}}$ is an MLWE encryption of $\mu \overline{s}_k g_j$ for all $0 \leq j < d$ and $0 \leq k \leq \text{idx}$. Finally, we show that the claim is true for $i > \text{idx}$, using mathematical induction again. Suppose that the claim is true for some $i = \iota$ where $\text{idx} \leq \iota$. Then, in the $\iota+1, j$ -th iteration of the outer loop, we compute the following:

1. $\text{acc}_{dk+j}^{\iota+1} \leftarrow (\text{acc}_{dk+j}^{\iota} \mid 0)$ for $0 \leq k \leq \iota$,
2. $\text{acc}_{d(\iota+1)+j}^{\iota+1} \leftarrow \sum_{k=0}^{\iota} \text{acc}_{dk+j}^{\iota} \boxtimes_v \text{cLEV}_{\mathbf{s}_{\iota+1}}(s_{\iota+1,k})$

Then, similar to the previous discussions, we obtain the following:

$$\text{acc}_{dk+j}^{\iota+1} \cdot \overline{\text{sk}}_{\iota+1} = [\text{acc}_{dk+j}^{\iota} \ 0] \cdot \begin{bmatrix} \overline{\text{sk}}_{\iota} \\ \mathbf{s}_{\iota+1}^{\top} \cdot \overline{\text{sk}}_{\iota} \end{bmatrix} = \text{acc}_{dk+j}^{\iota} \cdot \overline{\text{sk}}_{\iota} \approx \mu \overline{s}_k g_j \pmod{Q} \quad (13)$$

for all $0 \leq k \leq \iota$, and

$$\text{acc}_{d(\iota+1)+j}^{\iota+1} \cdot \overline{\text{sk}}_{\iota+1} \approx \sum_{k=0}^{\iota} \mu \overline{s}_k g_j \cdot s_{\iota+1,k} = \mu \overline{s}_{\iota+1} g_j \pmod{Q}. \quad (14)$$

From these two equations Eq. (13) and Eq. (14), it follows that the claim holds for $i = \iota + 1$ if it holds when $i = \iota$. We can finally deduce that the claim holds when $i = n$ as well, thanks to mathematical induction. Therefore, the output ciphertext is indeed an RGSW encryption of μ under $\overline{\text{sk}}_n$. $\square$

A.2 Proof of Lemma 2

Proof. By definition, $\mathbf{C} = [-\mathbf{a} \cdot \mathbf{s}^\top + \mu \cdot \mathbf{g} \otimes \mathbf{I}_k + \mathbf{E} \mid \mathbf{a}]$ for the randomness $\mathbf{a} \in R_Q^{kd}$ and the noise $\mathbf{E} \in R^{kd \times k}$. Then, we have the following:

$$\begin{aligned} \mathbf{e} &= \mathbf{c}' \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{s} \end{bmatrix} - \mu \cdot \mathbf{c} = h(\mathbf{c})^\top \cdot \mathbf{C} \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{s} \end{bmatrix} - \mu \cdot \mathbf{c} \\ &= h(\mathbf{c})^\top \cdot [-\mathbf{a} \cdot \mathbf{s}^\top + \mu \cdot \mathbf{g} \otimes \mathbf{I}_k + \mathbf{E} \mid \mathbf{a}] \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{s} \end{bmatrix} - \mu \cdot \mathbf{c} \\ &= h(\mathbf{c})^\top \cdot (\mu \cdot \mathbf{g} \otimes \mathbf{I}_k + \mathbf{E}) - \mu \cdot \mathbf{c} \\ &= h(\mathbf{c})^\top \cdot \mathbf{E} \pmod{Q}. \end{aligned}$$

Since $\|h(\mathbf{c})^\top\|_\infty \leq B_h$ and $\|\mathbf{E}\|_\infty \leq B_\sigma$, it directly follows that $\|\mathbf{e}\|_\infty \leq kdNB_\sigma B_h$. $\square$

A.3 Proof of Lemma 3

Proof. Similar to the correctness proof, we prove the claim using mathematical induction on the noise bound of acc_{dk+j}^i 's. Let us define the error function $\text{err}_{\overline{\mathbf{sk}}_i}(\cdot) : R_Q^{1 \times (i+1)} \rightarrow R$ for the MLWE encryptions acc_{dk+j}^i as follows:

$$\text{err}_{\overline{\mathbf{sk}}_i}(\text{acc}_{dk+j}^i) = \begin{cases} \text{acc}_{dk+j}^i \cdot \overline{\mathbf{sk}}_i - \overline{s}_k g_j & \text{if } i < \text{idx} \\ \text{acc}_{dk+j}^i \cdot \overline{\mathbf{sk}}_i - \mu \overline{s}_k g_j & \text{otherwise} \end{cases}$$

Let us begin by measuring the noise bound for $i < \text{idx}$ cases. Note that $\text{err}_{\overline{\mathbf{sk}}_i}(\text{acc}_{dk+j}^i) = \text{err}_{\overline{\mathbf{sk}}_k}(\text{acc}_{dk+j}^k)$, from the correctness proof of the algorithm. Therefore, we can deduce the following relation from the algorithm description and Lemma 2.

$$\begin{aligned} \text{err}_{\overline{\mathbf{sk}}_i}(\text{acc}_{di+j}^i) &= \text{acc}_{di+j}^i \cdot \overline{\mathbf{sk}}_i - \overline{s}_i g_j \\ &= \sum_{k=0}^{i-1} \left(\left(\text{acc}_{dk+j}^{i-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_i}(s_{i,k}) \right) \cdot \overline{\mathbf{sk}}_i - \overline{s}_k s_{i,k} g_j \right) \\ &= \sum_{k=0}^{i-1} \left(\left(s_{i,k} \cdot \text{acc}_{dk+j}^{i-1} + \mathbf{f}_{i,k} \right) \cdot \overline{\mathbf{sk}}_{i-1} - \overline{s}_k s_{i,k} g_j \right) \\ &= \sum_{k=0}^{i-1} \left(s_{i,k} \cdot \text{err}_{\overline{\mathbf{sk}}_{i-1}}(\text{acc}_{dk+j}^{i-1}) + \mathbf{f}_{i,k} \cdot \overline{\mathbf{sk}}_{i-1} \right), \end{aligned}$$

where $\mathbf{f}_{i,k}$ is the error vector from the vector gadget product $\text{acc}_{dk+j}^{i-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_i}(s_{i,k})$, i.e.,

$$\mathbf{f}_{i,k} = \left(\text{acc}_{dk+j}^{i-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_i}(s_{i,k}) \right) \cdot \begin{bmatrix} \mathbf{I}_i \\ \mathbf{s}_i \end{bmatrix} - s_{i,k} \cdot \text{acc}_{dk+j}^{i-1} \pmod{Q}.$$

Then, by Lemma 2 and Proposition 1, we have

$$\begin{aligned}
& \|\text{err}_{\overline{\mathbf{sk}}_i}(\text{acc}_{di+j}^i)\|_\infty \\
& \leq \sum_{k=0}^{i-1} \left\| \left(s_{i,k} \cdot \text{err}_{\overline{\mathbf{sk}}_{i-1}}(\text{acc}_{dk+j}^{i-1}) + \mathbf{f}_{i,k} \cdot \overline{\mathbf{sk}}_{i-1} \right) \right\|_\infty \\
& \leq \sum_{k=0}^{i-1} \left(H \cdot \|\text{err}_{\overline{\mathbf{sk}}_k}(\text{acc}_{dk+j}^k)\|_\infty + \frac{idN^2B_\sigma B_h}{H} ((H+1)^{i-1} + H - 1) \right) \\
& = \frac{i^2 dN^2 B_\sigma B_h}{H} ((H+1)^{i-1} + H - 1) + H \sum_{k=0}^{i-1} \|\text{err}_{\overline{\mathbf{sk}}_k}(\text{acc}_{dk+j}^k)\|_\infty
\end{aligned}$$

By mathematical induction, we obtain the norm bound of $\text{err}_{\overline{\mathbf{sk}}_i}(\text{acc}_{di+j}^i)$ for $1 \leq i < \text{idx}$, as follows.

$$\begin{aligned}
& \frac{dN^2 B_\sigma B_h}{H} \cdot \left(\frac{2H-2}{H} i + \frac{H^2 + H - 2}{H^2} + \right. \\
& \left. \left(\frac{H}{3} i^3 + \frac{H+2}{2} i^2 + \frac{H}{6} i + \frac{H^4 + 3H^3 + H^2 - 3H - 2}{H^2} \right) (H+1)^{i-2} \right)
\end{aligned}$$

For simplicity, we use the approximate bound of $\approx \frac{1}{3} dN^2 B_\sigma B_h i^3 (H+1)^{i-2}$. Note that this gives a fairly good asymptotic noise bound, especially considering that we are conducting a worst-case analysis. Now, let us focus on the case where $i = \text{idx}$. Firstly, by definition, it is easy to see that $\|\text{err}_{\overline{\mathbf{sk}}_{\text{idx}}}(\text{acc}_j^{\text{idx}})\|_\infty \leq B_\sigma$. Also, the noise bound above and Lemma 2 imply that

$$\|\text{err}_{\overline{\mathbf{sk}}_{\text{idx}}}(\text{acc}_{di+j}^{\text{idx}})\|_\infty \lesssim \frac{1}{3} \|\mu\|_1 dN^2 B_\sigma B_h i^3 (H+1)^{i-2} + \text{idx} \cdot dN B_\sigma B_h$$

for $1 \leq i < \text{idx}$. Finally, observe that the error term for $\text{acc}_{didx+j}^{\text{idx}}$ can be written as follows, similar to the case where $1 \leq i < \text{idx}$.

$$\begin{aligned}
\text{err}_{\overline{\mathbf{sk}}_{\text{idx}}}(\text{acc}_{didx+j}^{\text{idx}}) &= \sum_{k=0}^{\text{idx}-1} \left(\left(\mu s_{\text{idx},k} \cdot \text{acc}_{dk+j}^{\text{idx}-1} + \mathbf{f}_{\text{idx},k} \right) \cdot \overline{\mathbf{sk}}_{\text{idx}-1} - \mu \overline{s}_k s_{\text{idx},k} g_j \right) \\
&= \sum_{k=0}^{\text{idx}-1} \left(\mu s_{\text{idx},k} \cdot \text{err}_{\overline{\mathbf{sk}}_{\text{idx}-1}}(\text{acc}_{dk+j}^{\text{idx}-1}) + \mathbf{f}_{\text{idx},k} \cdot \overline{\mathbf{sk}}_{\text{idx}-1} \right),
\end{aligned}$$

where $\mathbf{f}_{\text{idx},k}$ is the error vector from the vector gadget product $\text{acc}_{dk+j}^{\text{idx}-1} \boxtimes_v \text{cLEV}_{\mathbf{s}_{\text{idx}}}(s_{\text{idx},k})$. Then, we can obtain the following bound for $\text{err}_{\overline{\mathbf{sk}}_{\text{idx}}}(\text{acc}_{didx+j}^{\text{idx}})$, in an analogous way to the $1 \leq i < \text{idx}$ case.

$$\begin{aligned}
\|\text{err}_{\overline{\mathbf{sk}}_{\text{idx}}}(\text{acc}_{didx+j}^{\text{idx}})\|_\infty &= \left\| \sum_{k=0}^{\text{idx}-1} \left(\mu s_{\text{idx},k} \cdot \text{err}_{\overline{\mathbf{sk}}_{\text{idx}-1}}(\text{acc}_{dk+j}^{\text{idx}-1}) + \mathbf{f}_{\text{idx},k} \cdot \overline{\mathbf{sk}}_{\text{idx}-1} \right) \right\|_\infty \\
&\leq \sum_{k=0}^{\text{idx}-1} \left(\|\mu\|_1 \cdot H \cdot \|\text{err}_{\overline{\mathbf{sk}}_k}(\text{acc}_{dk+j}^k)\|_\infty + \|\mathbf{f}_{\text{idx},k} \cdot \overline{\mathbf{sk}}_{\text{idx}-1}\|_\infty \right)
\end{aligned}$$

$$\begin{aligned}
&\leq \|\mu_1\| \cdot \sum_{k=0}^{\text{idx}-1} \left(H \cdot \|\text{err}_{\overline{\text{sk}}_k}(\text{acc}_{dk+j}^k)\|_\infty + \|\mathbf{f}_{\text{idx},k} \cdot \overline{\text{sk}}_{\text{idx}-1}\|_\infty \right) \\
&\lesssim \frac{1}{3} \|\mu\|_1 dN^2 B_\sigma B_h \text{idx}^3 (H+1)^{\text{idx}-2}
\end{aligned}$$

Finally, we cover the case where $i > \text{idx}$. By the discussion from the correctness proof, it is easy to see that $\text{err}_{\overline{\text{sk}}_i}(\text{acc}_{dk+j}^i) = \text{err}_{\overline{\text{sk}}_{\text{idx}}}(\text{acc}_{dk+j}^{\text{idx}})$. Therefore, the algorithm description and the above noise bounds imply that

$$\begin{aligned}
\|\text{err}_{\overline{\text{sk}}_i}(\text{acc}_{di+j}^i)\|_\infty &= \left\| \sum_{k=0}^{i-1} \left(s_{i,k} \text{err}_{\overline{\text{sk}}_{i-1}}(\text{acc}_{dk+j}^{i-1}) + \mathbf{f}_{i,k} \cdot \overline{\text{sk}}_{i-1} \right) \right\|_\infty \\
&\lesssim \frac{1}{3} \|\mu\|_1 dN^2 B_\sigma B_h (\text{idx}+1)^3 (H+1)^{\text{idx}-1} - \frac{\text{idx}^2 dN B_\sigma B_h}{H} ((H+1)^{\text{idx}-1} - 1) \\
&\quad + \frac{idN^2 B_\sigma B_h}{H} ((H+1)^{i-1} + H - 1) + H \cdot \sum_{k=\text{idx}+1}^{i-1} \|\text{err}_{\overline{\text{sk}}_{\text{idx}}}(\text{acc}_{dk+j}^{\text{idx}})\|_\infty.
\end{aligned}$$

By mathematical induction, we obtain the approximate noise bound

$$\frac{1}{3} dN^2 B_\sigma B_h i^3 (H+1)^{i-2} + \frac{1}{3} \|\mu\|_1 dN^2 B_\sigma B_h (\text{idx}+1)^3 (H+1)^{i-1}.$$

for $\text{idx} < i \leq n$. $\square$

A.4 Key-switching Key Generation

Lemma 4 (Key-switching Key Generation). *Given (fresh) cLEV encryptions $\text{cLEV}_{\mathbf{s}_i}(s_{i,j})$ for $1 \leq i, j \leq n$, and the index arrays idx and idx' corresponding to the joint keys $\overline{\text{sk}}$ and $\overline{\text{sk}}'$, respectively, let $\text{kSk}_{\overline{\text{sk}} \rightarrow \overline{\text{sk}}'}$ be the output of Algorithm 2. Then, for the noise vector $\mathbf{e} = (e_0, \dots, e_{dn-1}) := \text{kSk}_{\overline{\text{sk}} \rightarrow \overline{\text{sk}}'} \cdot \overline{\text{sk}}' - \mathbf{g} \otimes (\overline{s}_1, \dots, \overline{s}_n)^\top \pmod{Q}$, the following inequality holds,*

$$\|e_{di+j}\|_\infty \lesssim \frac{1}{3H^2} dN^3 B_\sigma B_h^2 (H+1)^{n+i-1} (n^3 + 8H).$$

for all $0 \leq i < n$ and $0 \leq j < d$.

Proof. Let us denote the noise from the external product $\text{acc}_{dk+j} \boxtimes \overline{\mathbf{C}}_{i,k}$ by $\tilde{e}_{i,j,k} := (\text{acc}_{dk+j} \boxtimes \overline{\mathbf{C}}_{i,k}) \cdot \overline{\text{sk}}' - s_{i,k} \cdot \text{acc}_{dk+j} \cdot \overline{\text{sk}}' \pmod{Q}$. Then, due to Lemma 3, we can bound the noise of the RGSW encryption and consequently obtain the following inequality:

$$\|\tilde{e}_{i,j,k}\|_\infty \lesssim \frac{(H+1)^{n-1}}{3H} dN^3 B_\sigma B_h^2 (n^3 + H(i+1)^3)$$

With this bound, we can devise a recurrence relation on the noise of acc_{di+j} 's. Recall that the noise e_{di+j} of the acc_{di+j} satisfies the equality $e_{d(i-1)+j} =$

$\text{acc}_{di+j} \cdot \overline{\mathbf{s}}\mathbf{k}' - g_j \overline{s}_i \pmod{Q}$. We also define $e_{-d}, \dots, e_{-1}$, which corresponds to the noise of $\text{acc}_0, \dots, \text{acc}_j$, respectively, for completeness. Then,

$$\begin{aligned}
\|e_{di+j}\|_\infty &= \left\| \left(\sum_{k=0}^{i-1} \text{acc}_{dk+j} \boxdot \overline{\mathbf{C}}_{i,k} \right) \cdot \overline{\mathbf{s}}\mathbf{k}' - g_j \overline{s}_i \pmod{Q} \right\|_\infty \\
&= \left\| \sum_{k=0}^{i-1} \left(s_{i,k} \cdot (\text{acc}_{dk+j} \cdot \overline{\mathbf{s}}\mathbf{k}' - g_j \overline{s}_k) + \tilde{e}_{i,j,k} \right) \pmod{Q} \right\|_\infty \\
&\leq H \sum_{k=0}^{i-1} \|e_{dk+j}\|_\infty + \sum_{k=0}^{i-1} \|\tilde{e}_{i,j,k}\|_\infty \\
&\leq H \sum_{k=0}^{i-1} \|e_{dk+j}\|_\infty + i \cdot \frac{(H+1)^{n-1}}{3H} dN^3 B_\sigma B_h^2 (n^3 + H(i+1)^3).
\end{aligned}$$

Since $\|e_{-d+j}\|_\infty = 0$ for all $0 \leq j < d$, we can use mathematical induction to obtain the claimed noise bound of

$$\|e_{di+j}\|_\infty \lesssim \frac{1}{3H^2} dN^3 B_\sigma B_h^2 (H+1)^{n+i-1} (n^3 + 8H)$$

for all $0 \leq i < n$ and $0 \leq j < d$.